Buying Time:
What Withdrawal Limits Can Save on Cross-Chain Bridges

Anonymous submission
Paper under double-blind review

***Abstract*—When a cross-chain bridge fails, how much of the money it holds can a withdrawal limit save? A 2024 systematization of interoperability expected limits to cut losses substantially and left this measurement to future work. In the 57 failures we collected, the funds left within minutes and the first response came hours later. A limit cannot tell an unauthorized withdrawal from a legitimate one, so it slows both and buys time for someone to pause the bridge. The party behind a failure can replay the busiest legitimate traffic, and a limit that delays no legitimate user must let that replay through. On the traffic of sixteen failed bridges, no such limit could have guaranteed more than 71% of the value at the observed response times. A check that reads both chains can tell whether a deposit backs each release. We show that if a contract holds the funds and every withdrawal waits 10 minutes while the check runs, no failure it flags loses anything. Run in shadow on two live bridges between Ethereum and BSC, the check found a deposit behind every one of their 13,261 releases in 42 days. The limit behind the check is the backstop for the check's own failure, and there a bucket fitted a month ahead keeps 72% of the value.**

1. Introduction

Money leaves in minutes. A cross-chain bridge holds assets on one chain and releases them when a message from another chain says so. The bridge's verifiers attest to each message, and a contract on the receiving chain checks the attestation before it releases the funds. If any of these steps fails, an unauthorized party can withdraw what the bridge holds. We collected the 57 failures of this kind recorded from 2021 to 2026, which released $3.7 billion (Section 2.3). Half of the outflows we could time were over within eight minutes of the first unauthorized release, and 68% of the value left within ten minutes. The first response, an acknowledgment or a pause, came after a median of 3 hours 42 minutes. Of the 19 recorded pauses, 17 came after the last release.

An open question. Bridges have answered with withdrawal limits. Augusto et al. [1] review the losses of eighteen bridge failures, recommend limits set from a bridge's usual flow, and leave their effect open: "We believe setting withdrawal limits or emergency pauses would significantly reduce this value. This empirical analysis remains a task for future endeavors" [2, p. 21]. Operators set the numbers by trial.

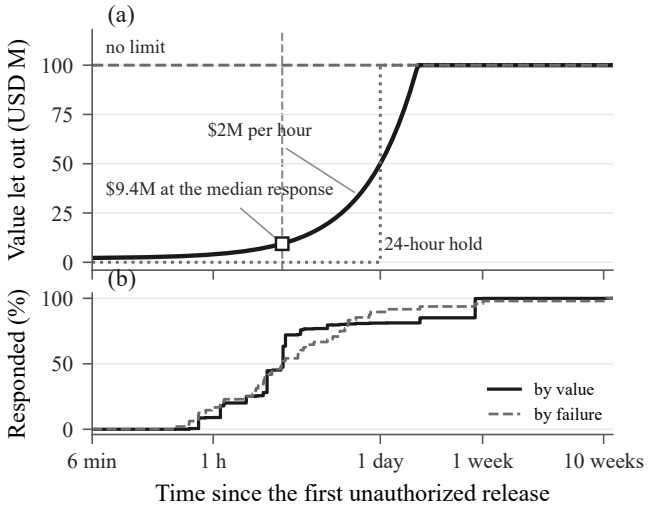


Figure 1. What a limit lets out, and when responses came. (a) A bridge that holds $100 million and whose users never withdraw more than $2 million in an hour: no limit (dashed), a token bucket at the release point that refills $2 million per hour and holds $2 million when full (solid; a straight line, bent by the logarithmic axis), and a 24-hour hold on every withdrawal (dotted). The vertical line is the median response time of our failures. (b) Share of the 48 failures with a recorded response whose first response came by each time, weighted by value and by failure; the second crosses 50% at the median of panel (a). Logarithmic time axis.

Asked about the limits of the bridge it rebuilt in 2022, the Ronin team answered: "We don't know the perfect numbers for the limits yet so, we will need to roll it out and measure it" [3, pp. 43–44]. This paper carries out that analysis: it measures what limits can save, where and at what cost to users, and shows how to set them.

A limit buys time. A limit cannot tell an unauthorized withdrawal from a legitimate one; if it could, the bridge would refuse the first. All it can do is slow every withdrawal down, and it saves money only if someone pauses the bridge before the limit has let the rest out. In Fig. 1, a limit of $2 million per hour lets out $9.4 million of $100 million by the median response time, and a 24-hour hold on every withdrawal lets out nothing if the response comes within the day and everything if it comes later. The loss is the height of the limit's curve at the moment of the response; we call this curve the limit's exposure curve. Where the limit sits and who controls it, its shape, its numbers and the response

time decide that height.

Where the limit sits. A failure lets withdrawals enter the bridge at some stage and skip the checks before it, and a limit acts only on withdrawals that pass through it. Taken keys and flawed receiving checks let withdrawals in after the verifiers, so a limit in the verifiers' own software sees only forged deposits on the sending chain, failures of the verifiers' own checks, and the redemption of claims minted without backing. On our failures, such a limit could have contained at most 27.0% of the value, whatever its parameters, because that is the share that passed the verifiers. A limit in the contract that releases the funds also sees taken keys and flawed receiving checks, if the verifiers can change neither it nor the pause (Section 4); where keys hold the funds directly, there is no such contract. A limit or a pause that the party behind the failure controls can be switched off, and none of the 22 limits we read in source code makes a loosening of its parameters wait.

What shape it has. Token buckets and windows are ramps, letting out value in proportion to time; a hold on every withdrawal, or a queue that releases itself, is a cliff, letting out nothing until a timer runs out and everything after (Section 5). Every limit that restores its allowance by itself lets everything out if no response comes. A ramp's loss follows the mean response time, and a cliff's loss follows the tail of the response time. A limit that nets deposits against withdrawals adds the other side's limit to its own. A hold on large withdrawals contains nothing by itself against a party that splits them; what it buys is a tighter limit for the rest.

What numbers to use. Within any time d after a failure, no limit that leaves legitimate users undelayed can let out less than the most that legitimate users ever withdrew within d, since the party can replay that traffic. Network calculus gives this floor and the limit that attains it [4], [5]. The floor binds every defense that sees only the amounts and times of withdrawals, including an alarm on the outflow's size or pace. On the traffic of the sixteen failed bridges that carried 89% of the value (Section 7.1), no limit that delays none of a bridge's transfers in the 150 days before its failure could have guaranteed more than 71% at the observed response times. Most of the value lost is the 2022 Ronin failure, answered after six days; in the busiest six days of its prior traffic an exchange had withdrawn $722 million in two transfers, more than the failure took. A limit fitted a month ahead must delay someone as traffic grows. A bucket refitted monthly for a six-hour response keeps 72% while delaying 17% to 48% of legitimate value; about half of that delay is unavoidable (Proposition 7.2).

With a check. A check that a deposit backs each release escapes the floor, and the limit behind it is the backstop when the check fails. Comparing the value released with the value deposited, the check flags every failure whose withdrawals enter at the verifiers or later, and a forged deposit too, since a forged deposit event moves no value. Used as a gate that holds every withdrawal for 10 minutes while it runs, it stops every failure it flags if a contract holds the funds (Section 8). If the check fails, the gate alone keeps nothing and the bucket behind it still keeps 72%. Sending only the largest transfer in a thousand to the check spares 99.8% of transfers the wait and keeps 78%, but 8% if the check fails, since the party's large withdrawals then pass. Such a check paused a testbed bridge within a minute of each injected failure, forged deposits included (Section 11), and found a deposit behind each of the 13,261 releases of two live bridges between Ethereum and BSC in 42 days (Section 8).

What the stake must cover. With a limit downstream of the verifiers, and a pause and a confiscation decision they do not control, their stake needs to exceed only what the limit lets out before the response, not the value the bridge holds (Section 9).

Contributions.

- A dataset of 57 failures coded by entry stage, control and response and timed where the sources allow, recoded blind from a codebook, and a source-code reading of 22 limits (Section 2, Appendix B).
- A compact calculus built on network calculus: placement, the exposure curves of deployed limits, a floor set by legitimate traffic that binds every defense seeing only amounts and times, the least delay at a given exposure, and what a check from outside the releases adds (Sections 3–7).
- Counterfactuals on the failed bridges' own traffic, in sample and a month ahead, with and without the check and with its failure, at their cost to users, on CCIP and on Wormhole, and a consistency check against the 13 failures that met a deployed limit (Sections 6–8).
- Rules for operators, two prototype limiters, an end-to-end testbed of the check and the pause, and a shadow run of the check on two live bridges (Sections 8, 10 and 11).

2. Bridges, limits and failures

2.1. How a bridge releases funds

A transfer passes four stages (Fig. 2). A user locks or burns an asset on the sending chain, and a contract there emits an event. The bridge's verifiers attest to the event; depending on the bridge they are a validator set, a group of signers, an oracle network or a light client. A contract on the receiving chain checks the attestation and the message. Last, that contract releases the funds: it withdraws an asset from an escrow or a liquidity pool, or it mints a wrapped asset. We call this last stage the release point. A lock-and-mint bridge releases by minting in one direction and from escrow in the other; a liquidity bridge releases from a pool in both. A bridge has many routes, one for each sending chain, release contract and asset.

2.2. Deployed limits

We read the source code of 22 limits that bridges, messaging protocols and token standards deploy or propose,

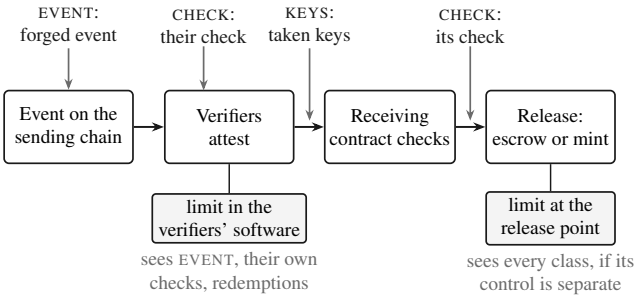


Figure 2. The four stages of a transfer, and where each class of failure lets unauthorized withdrawals in; a CHECK failure enters after the check that failed, the verifiers' own (left) or the receiving contract's (right). A limit sees a failure only if the failure's withdrawals pass it and the failing party cannot change it: a limit in the verifiers' software sees EVENT failures, failures of their own checks and the redemption of unbacked mints, and a limit at the release point sees every class if its control is separate from the verifiers.

each at a fixed commit (Table 1); we call each of them a rule. Five properties decide what a rule contains: the stage where it runs, the shape of what it lets out over time, whether it nets inflows against outflows, whether queued withdrawals leave by themselves when a timer runs out, and who controls its parameters and its pause. Of the rules, four run in the verifiers' software, one acts only on the sending chain, thirteen act where funds are released (or at the token's mint) under a party other than the verifiers, three more, Axelar's two and Ronin's, have parameters that the verifiers partly govern, and Circle's CCTP is controlled by its verifier. Every on-chain pause takes effect at once. No rule's code delays a loosening of its parameters: a change takes effect at once, or with a node release (Governor) or an upgrade (StarkGate), and the only timelocks sit in the governance above the CCIP and Axelar contracts, each with a path that skips them. Five rules have no pause on releases.

The census of CCIP lanes. Of the 2,992 token lanes configured on Chainlink CCIP [6], 2,021 have an outbound token bucket, 1,986 of these also an inbound bucket in the releasing token pool, and the median outbound bucket holds $623 thousand and refills in 6.0 hours.

2.3. Failures

We built a dataset of failures from DefiLlama's list of incidents [7], the operators' post-mortems and the chains themselves (Appendix B, with the 20 largest in Table 9). Of the 69 records of at least $1 million since 2021 that DefiLlama flags as bridge incidents or classes as cross-chain, or that the 2024 systematization lists [1], 57 are unauthorized withdrawals from a bridge's escrow, minting or liquidity, together $3,711 million. We coded each failure by the stage where its withdrawals entered (Fig. 2): a forged deposit event on the sending chain (EVENT, 6 failures, 11% of the value), control of the keys that authorize releases (KEYS, 21, 32%), a flaw in a check, in the receiving contract or the verifiers' own software, that accepted a message matching no deposit (CHECK, 25, 57%), and the rest, such as accounting flaws within one chain (OTHER, 5, 0.3%). By the side that released, 63% of the value left an escrow or a pool, 19% was minted, and 18% was minted on one chain and redeemed from an escrow on another. 20 failures saw value returned or frozen later, 47% of the total; we count released value gross and report the net share where it matters. A blind coder, a separate language-model session given a codebook, our descriptions of each failure with the codes removed, and the public sources, agreed on the class of 55 of the 57 failures (Cohen's $\kappa = 0.95$); the disagreements carry 0.4% of the value.

We read the times of the first and the last unauthorized release from the chains, and the times of the first public alert by a third party, the operator's first acknowledgment and the pause from the operators' posts or the chains. The first response is the earlier of the acknowledgment and the pause; 9 failures, with $39 million in all, have none recorded.

Table 2 gives the timing. The releases end within minutes: half of the outflows lasted less than eight minutes, and 68% of the value left within ten minutes of each failure's first release. The responses came hours later, and 17 of the 19 pauses came after the last release. Without a limit, a pause had almost nothing left to save.

3. Model

Limits. A limit Λ sits at one stage of one or more routes and acts only on the withdrawals that pass it. At each moment it decides whether to pass each withdrawal on, keep it queued or reject it, using only what happened before. It cannot tell an unauthorized withdrawal from a legitimate one.

Failures. A failure at time τ lets an unauthorized party submit withdrawals that enter at one stage and skip the stages before it, on a set of routes we call its scope, in any amounts and at any times after τ. An EVENT failure enters before the verifiers, a KEYS failure after them, and a CHECK failure after the check that failed, the verifiers' own or the receiving contract's. The party can take at most W: the value held on the routes in scope, or what the claims it can mint are worth on the receiving chain.

Control and response. A limit or a pause that the party controls through keys or governance rights counts as absent, unless changing it takes longer than the response (Proposition 4.4). At time $\tau + D$ a party independent of the failure pauses the release and cancels every queued withdrawal. We call D the response time; if no independent party holds a pause, $D = \infty$. The loss is the unauthorized value released in $[\tau, \tau + D)$.

Legitimate traffic. Let $\mathcal{T}$ be the set of possible legitimate withdrawal histories of the bridge, described by their amounts and times. It is closed under shifts in time and under dropping a prefix, so any stretch of a legitimate history, started at any time, is again legitimate. Its envelope $E(u)$ is the largest legitimate value withdrawn within any window of length u. A limit has delay h if, from its quiet

TABLE 1. LIMITS, READ IN SOURCE CODE AT FIXED COMMITS. STAGE: VERIFIERS, IN THE VERIFIERS' SOFTWARE; SEND, ON THE SENDING CHAIN; RELEASE, IN THE CONTRACT THAT RELEASES ON THE RECEIVING CHAIN; TOKEN, IN THE TOKEN CONTRACT; SEND + RELEASE, A LIMIT ON EACH SIDE. SHAPES ARE THOSE OF SECTION 5. ACCOUNTING: GROSS COUNTS RELEASES ONLY; NET SUBTRACTS COUNTED INFLOWS (PROPOSITION 5.3). AUTOMATIC RELEASE: QUEUED WITHDRAWALS LEAVE WHEN A TIMER EXPIRES; UNCOUNTED MEANS OUTSIDE THE LIMIT. PARAMETERS: WHO CAN CHANGE THE LIMIT; NO RULE'S CODE DELAYS A LOOSENING. PAUSE: WHO CAN STOP RELEASES. ISSUER: THE TOKEN'S ISSUER; OPERATOR, OWNER AND GOVERNORS: A PARTY OTHER THAN THE VERIFIERS.

Rule	Stage	Shape	Accounting	Automatic release	Parameters	Pause
Wormhole Governor	verifiers	cliff at 24 h (sliding 24 h cap before)	gross (net with flow cancelling)	yes, uncounted	verifiers	any 7 of the 19 verifiers (withhold)
Wormhole Global Accountant	verifiers	bounded (releases up to recorded locks)	conservation ledger	no	verifiers	none
Wormhole NTT	send + release	ramp; cliff for queued items after 24 h	net (deposits of the same token)	yes, uncounted	issuer	issuer
Chainlink CCIP token pools	send + release	ramp (token bucket)	gross	no	issuer	CCIP owner (curse)
LayerZero OApp RateLimiter	send	ramp	optional net	no	app owner	app owner
LayerZero stablecoin and RWA OFT	send + release	ramp	optional net (deposits of the same token)	no	issuer	sends only
Axelar ITS	send + release	ramp (6 h calendar windows)	net within window (deposits of the same token)	no	issuer or operator	operator
Axelar legacy gateway	release	ramp (6 h calendar windows)	gross	no	Axelar governance	pauser or governance
Hyperlane rate-limited hook and ISM	send or release	ramp	gross	no	route owner	route owner, if added
Hyperlane net-flow and delayed-flow	send + release	geometric; cliff at maxDelay for delayed flow	net (deposits of the same token)	delayed flow: yes	route owner	route owner, if added
Circle CCTP	send; token	per-message cap at source; bounded at the token	n/a	n/a	verifier (Circle)	verifier (Circle)
Linea	send + release	ramp (window from first use; ETH only)	gross	no	operator	operator; council
StarkGate	send + release	geometric (5% of balance per day)	gross within the day	no	operator	none on releases
Ronin	release	ramp (UTC day); bounded above the lock threshold	gross	no	bridge admin	admin; pauser
Celer cBridge	release	ramp plus delay for large items	gross	yes, counted when queued	governors	pausers
Gnosis OmniBridge and xDai	send + release	ramp (UTC day)	gross	no	owner	none (limit to zero)
BNB Chain BEP-171	release	cliff at 12 h	n/a	yes	issuer; governance	validators
xERC20	token	ramp	gross	no	issuer	none (limit to zero)
Taiko QuotaManager	release	ramp	gross	no	rollup owner	owner; pauser
THORChain outbound delay	verifiers	ramp then cliff	n/a	yes	verifiers	one node
THORChain solvency halt	verifiers	bounded (wallet against ledger)	wallet against ledger	auto-resume when solvent	verifiers	automatic
ERC-7265 (draft)	release	geometric; cliff in hold mode	net (reported inflow)	hold mode: yes	protocol admin	protocol admin

TABLE 2. TIMING OF THE FAILURES, MEASURED FROM THE FIRST UNAUTHORIZED RELEASE: MEDIAN AND QUARTILES OVER THE n FAILURES WHERE EACH TIME IS KNOWN. LAST RELEASE, 90% OUT: A QUARTILE OF 0 S MEANS THAT AT LEAST A QUARTER OF THESE FAILURES RELEASED EVERYTHING WITHIN ONE BLOCK. ALERT: THE FIRST PUBLIC ALERT BY A THIRD PARTY. ACKNOWLEDGMENT: THE OPERATOR'S FIRST PUBLIC STATEMENT. FIRST RESPONSE: THE EARLIER OF THE ACKNOWLEDGMENT AND THE PAUSE, OR FOR ONE FAILURE A THIRD PARTY'S REPORT THAT THE PAUSE HAD HAPPENED.

Quantity	n	Median	Quartiles
Last release	48	7 min 52 s	0 s to 1 h 8 min
90% of the value out	34	10 s	0 s to 11 min 31 s
Alert by a third party	26	1 h 25 min	53 min 24 s to 4 h 2 min
Acknowledgment	47	5 h 0 min	2 h 35 min to 12 h 22 min
Pause	19	3 h 39 min	1 h 0 min to 8 h 9 min
First response	48	3 h 42 min	2 h 3 min to 11 h 23 min

state after a long period without withdrawals, it releases every withdrawal of every history in $\mathcal{T}$ within h of its arrival.

Two measures. The *exposure curve* $\beta_\Lambda(d)$ is the most that Λ can be made to release within a time d after a failure that it sees, capped at W. It is taken over all strategies of the unauthorized party, from the limit's most permissive state, with no legitimate withdrawal competing for its allowance; the party chooses when to start, so it can wait for that state. For every rule of Table 1 the most permissive state is the quiet state: a full bucket, an empty window. The *delay cost* of Λ is which legitimate withdrawals wait, and for how long.

One formula. For a failure answered after a time D, the loss satisfies

$$L(D) \le \min\{W, \beta(D)\}, \tag{3.1}$$

where β is the exposure curve of the limits that the failure's withdrawals pass and that its party cannot switch off; for token buckets, sliding windows and holds the bound is attained (Theorem 4.1). For a failure that released a value a, a limit contains the share $1 - L/a$.

What the party can do. Besides choosing when to start, the party can split its withdrawals, spread them over the routes

in scope, and send deposits through the bridge (Sections 4 and 5). Until Section 8, the way it withdraws does not change the response time, and throughout, it cannot move the prices a limit uses (Proposition A.3 prices a wrong one).

4. Where a limit must sit

A limit protects against a failure only if the failure's withdrawals pass through it and the failing party cannot switch it off. In a row of limits every withdrawal passes each of them. Network calculus describes such a row: with the min-plus convolution $(f \otimes g)(d) = \inf_{0 \le s \le d}[f(s)+g(d-s)]$, limits with curves f and g in series let out at most $(f \otimes g)(d)$ within d, and a hold for a time H has the curve δ_H, which is 0 up to H and $+\infty$ after [5]. Call a limit *active* for a failure if it lies on a route in the failure's scope at or after its entry stage, the party does not control it, and it is a hold or lets out at most $\beta_j(u)$ within any interval of length u.

Theorem 4.1 (Placement; concatenation of shapers [4], [5])**.** *Consider a failure with scope $\mathcal{R}$. (i) If some route in $\mathcal{R}$ passes no active limit, then $L(d) = W$ for every $d > 0$. (ii) If every route in $\mathcal{R}$ passes at least one active limit of a set $\mathcal{C}$, then $L(d) \le \min\{W, \sum_{j \in \mathcal{C}} \beta_j(d)\}$. (iii) On a single route with active limits $\beta_1, \dots, \beta_k$ in series,*

$$L(d) \le \min\{W, (\beta_1 \otimes \cdots \otimes \beta_k)(d)\} \le \min\{W, \min_j \beta_j(d)\}. \tag{4.1}$$

The first bound is attained when the limits are token buckets, sliding windows or holds, and the second is an equality when the curves are concave.

The proofs are in Appendix A, where we also check that every rule of Table 1 meets the hypothesis. The second bound in (4.1) can be strict: a cap of one unit per period followed by a cap of two units per three periods lets out at most $1 + 2 = 3$ units within four periods (one period of the first cap, then one window of the second), while each cap alone would let out four.

Corollary 4.2 (Per-channel and pooled limits)**.** *Let each channel k (a chain's release contract, or an asset) pass its own active limit β_k and a pooled active limit β_Σ common to all. A failure whose scope covers a set K of channels loses at most $\min\{W, \beta_\Sigma(d), \sum_{k \in K} \beta_k(d)\}$ within d.*

A pooled limit stops failures that reach many channels at once, and per-channel limits stop failures confined to one; in series, each stops the failures it is built for. A pooled limit must sit where every channel's releases pass. Where they leave one contract, such as a token pool that serves many lanes or an escrow that holds many assets, that is the release point. Chains that release from separate contracts can share a pooled limit only through state that the verifiers carry, a stage that misses most failures.

Observation 4.3 (Mint side)**.** *A limit only on the escrow of the sending chain does not see an unauthorized mint on the receiving chain. The party sells the minted claims there, and the loss is W, their value on the receiving chain; only claims redeemed through the escrow pass the limit.*

A limit can sit downstream of every failure and still be absent if the failing party can change it, as most deployed rules let their owner do at once (Section 2.2).

Proposition 4.4 (Timelocked control)**.** *Let the party control a limit's parameters, but let a change that loosens them take effect only a time T after it is announced, while the response cancels pending changes. Against this party, the limit's exposure curve is $\beta(d)$ for $d \le T$ and W for $d > T$: a cliff at T on top of the limit's own curve. If $D \le T$, the loss is that of a limit the party does not control.*

Deployed placements. The Wormhole Governor is a signing policy in the software of the guardians, Wormhole's verifiers [8]: it sees a forged deposit, but a flawed receiving check bypasses the guardians, and a quorum of guardians whose keys are taken signs without it. A CCIP token pool's inbound bucket sees all three classes, under parameters the token's administrator sets; CCIP's curse, which stops releases, is set by hand by a multisig of Chainlink node operators and Chainlink Labs, an independent pause as far as those signers are not the committee that attests to messages, which we did not check.

Replay. We replayed each shape of rule at two placements on the 48 failures with a recorded response, which released $3,672 million (Fig. 3). For a failure that released a and was answered after D, the loss under a rule is $\min\{a, k\beta(D)\}$ if the rule sees the failure and a otherwise, where k is the number of chains the failure drained, each with its own copy of the rule (Corollary 4.2); Section 7.1 fits the rules to each bridge's own traffic. A rule in the verifiers' software sees the EVENT failures, which carried $406 million, the five CHECK failures whose flawed check ran in the verifiers' own software, and the redemption leg of the three unbacked mints redeemed through the same bridge, most of the 2022 Wormhole mint among them. A rule at the release point sees the other classes too, except 2 CHECK failures whose flaw would also have bypassed a limit in the releasing contract, such as an upgrade that removed its checks. We assume, as Section 10 recommends, that a party other than the verifiers sets its parameters and holds the pause. As the rules were actually governed, 12 of the 21 KEYS failures (10.9% of the value) used keys that also controlled the releasing contract, and the rule would not have seen them. In 8 failures, 17% of the value, keys held the custody directly, with no contract that could hold a limit; there a limit can run only in the signers' software, which sees what a verifier-side limit sees.

Whatever its shape and parameters, a rule in the verifiers' software would have contained at most 27.0% of the value. At the release point, a token bucket of $1 million per hour would have contained 87.2% (80.6% as governed, 82.8% with custody held by keys limited in the signers' software) and a 24-hour hold 80.7% (73.7%; 76.9%). The verifier-side ceiling is the share that passed the verifiers, so it rests on the coding: under the readings that the post-mortems leave open, it lies between 18.2% and 27.4% (Appendix B), the blind coder's reading of the legs gives 27%, and without the three largest failures, none of which passed the verifiers,

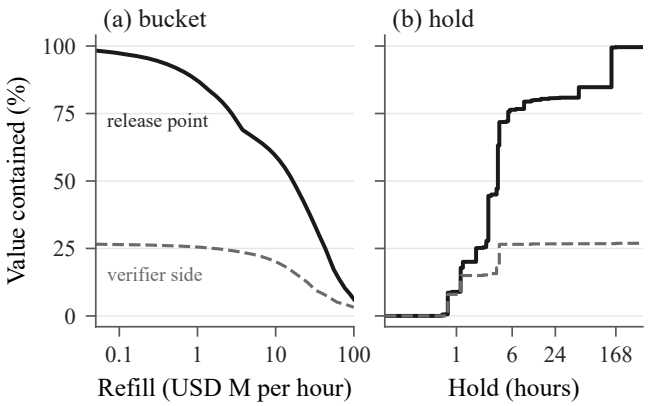


Figure 3. Share of the value of the 48 failures with a recorded response that a rule would have contained, at the release point (solid) and in the verifiers' software (dashed), with the response times as observed. (a) Token bucket with refill r and a capacity of one hour of refill, one for each chain a failure drained. (b) Hold on every withdrawal for H, cancelled by the response.

TABLE 3. EXPOSURE CURVES OF THE DEPLOYED FAMILIES FROM THE MOST PERMISSIVE STATE, CAPPED AT W. $N(d)$: MESSAGES PROCESSED WITHIN d; $n(d) = \lceil d/w \rceil + 1$: CALENDAR WINDOWS TOUCHED WITHIN d; β_0: THE RULE'S CURVE UNDER AN AUTOMATIC RELEASE AFTER A; β_B: THE CURVE ON THE SIDE THAT PAYS DEPOSITS BACK.

Rule	$\beta(d)$	Shape
Cap C per message	$C\,N(d)$	throughput
Sliding window, C per w	$C\lceil d/w \rceil$	ramp
Calendar window, C per w	$C\,n(d)$	ramp
Token bucket (b, r)	$b + rd$	ramp
Hold H, cancellable	0 for $d \le H$, else W	cliff
Hold H, then bucket	$(b + r(d - H))\,\mathbf{1}[d > H]$	ramp
Automatic release after A	$\beta_0(d)$ for $d \le A$, else W	cliff
Share p of balance per window	$W\big(1 - (1-p)^{n(d)}\big)$	geometric
Bucket netting deposits	$b + rd + \beta_B(d)$	ramp
Net flow, fabricated inflows	W	immediate
Cumulative cap θ, reviewed	θ	bounded

it is 51.2%.

5. What shape a limit has

The shape of the exposure curve decides how the loss grows with the response time (Table 3).

Proposition 5.1 (Deployed families)**.** *Each rule of Table 3 has the exposure curve listed there.*

A calendar window lets its full cap out just before a boundary and again just after it, so its curve starts at twice the cap. A cap per message without a rate limit is bounded only by how fast messages are processed. The last column has six shapes; Observation 5.2 and Proposition 5.3 explain why only one of them is bounded.

Observation 5.2 (Regeneration)**.** *Suppose a rule restores a positive allowance after any period without releases, with no action by a person or by a party independent of the failure: by a refill, a window that slides or resets, a timer that releases a queue, or a new window of a balance share. If no response comes, then* $\sup_d \beta(d) = W$.

Every ramp, cliff and geometric rule therefore buys time and nothing more. A bound that does not depend on the response time needs a person or an independent check in the loop, as in a cumulative cap that resets only after a review, which stops legitimate traffic each time it has moved θ.

Proposition 5.3 (Net flow across both sides)**.** *Let a rule on side A of a bridge let out within d at most $\beta_A(d)$ plus the inflows $I(d)$ it counts. (i) If each counted unit of inflow costs the party c in value it does not get back, its take within d is at most* $\min\{W, \beta_A(d) + (1-c)I(d)\}$*: as tight as β_A for $c \ge 1$, and growing with I for $c < 1$. (ii) If the counted inflows are deposits of the same token that the bridge pays back to the party on side B through active limits with curve β_B, the take is at most* $\min\{W, \beta_A(d) + \beta_B(d)\}$*, and the party can reach it without capital of its own, up to the time a deposit takes to be paid out.*

Of the six rules besides the Governor that net inflows, four count deposits of the same token. Axelar's flow limits and the LayerZero stablecoin option reject the matching release above the other side's limit, so they add it; NTT releases the matching transfer after 24 hours without counting it, and Hyperlane's net-flow rule does not limit the matching mint, so there the round trip is bounded only by the party's time. The Governor's flow cancelling counts transfers that the guardians attest, which a EVENT failure on the partner chain fabricates at no cost ($c = 0$), up to that chain's own cap.

Proposition 5.4 (Holds by observable class)**.** *Let a rule hold until a review every withdrawal in a class $\mathcal{K}$ defined by observable features, such as a size above s, a destination or an anomaly score, and pass the others through a limit Λ. (i) Against a party that can keep its withdrawals outside $\mathcal{K}$ at no cost, the rule's exposure curve is β_Λ. (ii) Λ needs to pass only legitimate traffic outside $\mathcal{K}$, so with delay h it can be as tight as $E_{\bar{\mathcal{K}}}((d-h)^+)$, where $E_{\bar{\mathcal{K}}}$ is the envelope of that traffic. (iii) If held withdrawals are released by a timer after A instead of a review, the curve is* $\max\{\beta_\Lambda(d), W\,\mathbf{1}[d > A]\}$*, a cliff.*

A hold on large withdrawals therefore contains nothing by itself against a party that can split below s at no cost; what it buys is the tighter limit for small ones. Sections 7.1 and 7.2 measure that gain.

Observation 5.5 (Mean and tail)**.** *Let the response time D have distribution G. A token bucket that lets out less than W within every response time loses $b + r\,\mathbb{E}_G[D]$ on average, and a hold for H loses $W\,\Pr_G(D > H)$.*

Replay. At the release point, a token bucket would have contained 97.3% of the value with a refill of $0.1 million per hour and 35.0% with $30 million per hour (Fig. 3). A hold would have contained 8.9% with a deadline of one hour, 76.4% with six hours and 80.7% with a day; a longer deadline adds little (84.8% at three days), because

TABLE 4. FAILURES THAT MET A RULE IN FORCE. PREDICTION: WHAT THE MODEL SAYS THE RULE COULD HOLD, FROM ITS PLACEMENT, CONTROL AND SHAPE (SECTIONS 4 AND 5); FOR THE FIRST THREE RULES, AT THE RELEASE POINT UNDER SEPARATE CONTROL, THE BOUND FROM THE PARAMETERS IN FORCE AND THE TIME THE OUTFLOW RAN. OUTCOME: WHAT THE FAILURE RELEASED (APPENDIX B).

Failure	Rule in force	Prediction	Outcome
Taiko (2026)	ether quota of 1,000 a day, 999 used before	at most 297 ether by the last release (41.7 an hour)	278 ether (94%); assets without a quota in full
Ronin (2024)	review above 4,000 ether or 2M USDC; daily cap	at most 4,397 ether and 2.48M USDC	3,996 ether (91%), 2.00M USDC (80%)
Poly Network Neo (2024)	a fifth of the backing in the releasing contract	at most that contract's balance	ended when it was empty
Nomad (2022)	30-minute window on new message roots	a hold in a skipped stage holds nothing	root accepted at initialization
THORChain (2026)	halt of the verifiers' signing	a halt there misses key-signed releases	key-signed releases went on
QANplatform (2022)	cap per transaction for each signer	a cap the failing key controls is absent	cap set to the maximum
ChainSwap (2021)	quota per signer	a cap per message does not bound the total	1,136 releases in 35 minutes
Maya Protocol (2026)	delay for large outbound transfers	a delay by size holds nothing below it	no outbound was delayed
AFX Trade (2026)	200-second dispute window	a cliff holds nothing after its deadline	elapsed unchallenged
Aztec 2.0 (2026)	escape window of 48 minutes in 16 hours	an open window without a cap holds nothing	all releases in an open window
NEAR Intents (2026)	outlier monitor that pauses	an alarm alone saves what is left	paused after the last release
Shibarium (2025)	halt on inconsistent checkpoints	an alarm alone saves what is left	halted after both releases
Nomic on Osmosis (2026)	static cap on a pool's share	a cap whose room returns lets all out	18 of 40.65 on day one; the rest later

most of the late value is the 2022 Ronin failure, answered after six days. These shares keep each response time as observed. A hold also postpones the outflow through which 20 failures were found; if their response came only after the hold released them, the 24-hour hold would contain 64.9%. Weighted by failure, the hold contains 79% and the bucket 32%: a hold stops small failures as fully as large ones, a ramp mostly the large. Net of the value later returned or frozen, the bucket contains 79.2% and the hold 67.6%.

6. Failures that met a limit

Of the 57 failures, thirteen met a rule in force that limits, holds or pauses releases. Table 4 gives what the model predicts from the rule's placement, control and shape, and what happened. We wrote the predictions knowing the outcomes, so this checks consistency rather than testing the model; two predictions are amounts that the chain can check.

The Taiko bridge gave ether a quota of 1,000 a day that refilled at 41.7 an hour. Three hours before the first release, a processed message counted 999 ether against the quota but moved none, so with the refill the model bounds the ether released by the last release, four hours after the first, at 297; the contract's own reading just before the first release, 128.22 ether available, matches the refill formula. 278 ether left in small releases at the pace of the refill, 94% of the bound and never above it; assets without a quota left in full. In the 2024 Ronin failure, withdrawals above 4,000 ether or 2 million USDC were locked for review, and those below counted against a daily cap of 4,500 ether and 2.5 million USDC, partly used that day, so a party that avoids the lock takes at most 4,397 ether and 2.48 million USDC. It took 3,996 ether (91%) and 2.00 million USDC (80%), one withdrawal each just below the lock size, as Proposition 5.4 predicts. Poly Network's Neo route kept about a fifth of its backing in the contract that released, and the outflow on Ethereum ended when that contract was empty. The other ten predictions follow from placement and control alone: the rule sat in a stage the failure skipped or under the failing key, held nothing below its size, or gave out before the response, as Observation 5.2 predicts for the cap on Osmosis.

7. Setting the numbers

A limit's numbers trade the loss at the response time against the delay of legitimate users, and legitimate traffic sets a floor under the trade.

Theorem 7.1 (Floor; minimal arrival curve and greedy shaper [4], [5])**.** *(i) If a limit Λ has delay h, then $\beta_\Lambda(d) \geq \min\{W, E((d-h)^+)\}$ for every d. The limit Λ^* that holds each withdrawal for h and then releases greedily, so that no window of length u releases more than $E(u)$, has delay h and $\beta_{\Lambda^*}(d) = \min\{W, E((d-h)^+)\}$. (ii) The bound in (i) holds for any defense that decides when to release and whether to pause using only the amounts and times of the withdrawals it receives and of its own releases, such as a limit with an alarm on the size or pace of releases, if from its quiet state it releases every history in $\mathcal{T}$ within h of each arrival and pauses none. If instead it pauses each history in $\mathcal{T}$ within d with probability at most α, its expected loss within d is at least $(1-\alpha)\min\{W, E((d-h)^+)\}$. (iii) If a party independent of the failure pauses within ℓ of the first withdrawal that no deposit backs, the loss with a limit Λ is at most $\min\{W, \beta_\Lambda(\min\{d, \ell\})\}$. If every withdrawal is first held for $H \geq \ell$, such a failure loses nothing, and one whose withdrawals the check passes loses at most $\min\{W, \beta_\Lambda((d-H)^+)\}$.*

The unauthorized party can replay the busiest legitimate window, and a limit that lets every legitimate history through within h must let that replay through as well; so must any defense that sees only amounts and times, since the replay looks legitimate to it, and a detector that pauses at random escapes only by its false-alarm probability. A defense that also uses what the party cannot copy, such as the history of recipients or the call structure of a transaction, as transaction-level detectors do [9], [10], escapes the floor as far as those features separate the two, at the cost of the legitimate withdrawals it flags. Information from outside the releases, the deposits on the sending chain, escapes it entirely; part (iii) gives the price, a hold on every withdrawal,

and Section 8 evaluates it. With $h = 0$, no limit that never delays a legitimate user can lose less than $E(D)$, the most that legitimate users ever withdrew within a response time. A failure smaller than that passes under every such limit, so limits protect against large failures. Since Λ^* attains the floor at every d at once, it has the smallest loss among limits of delay h for every distribution of response times.

Proposition 7.2 (Delay budget)**.** *Let a defense as in Theorem 7.1(ii) let out at most x within any interval of length d. On every legitimate history ℓ, it delays by more than h a value of at least $F(x) = \max \sum_k (\ell(I_k) - x)^+$, the maximum over disjoint intervals I_k of length $d - h$.*

A limit that lets out less than the floor of (i) must therefore delay users, and Proposition 7.2 says by how much at least. A sliding window of length w with cap $E(w)$ delays no legitimate history but lets out $E(w)$ within any $d \le w$ (Corollary A.1): a daily cap set at the busiest legitimate day lets out $E(24\,\mathrm{h})$ in a failure answered within an hour.

Proposition 7.3 (One token bucket; burstiness curve [11])**.** *A token bucket with capacity b and refill r has delay zero if and only if $b \ge B(r) = \sup_u [E(u) - ru]$. Among these buckets the smallest release within D is $\hat{E}(D) = \min_r [B(r) + rD]$, where $\hat{E}$ is the least concave majorant of E, and the bucket whose line supports $\hat{E}$ at D attains it.*

A bucket's parameters should therefore follow the response time it is meant for. Averaged over uncertain response times, the bucket that supports $\hat{E}$ at the mean lets out the least (Proposition A.2), but the loss is capped at W, and when W binds for long responses the best bucket supports $\hat{E}$ at a shorter time: on Wormhole's combined traffic with $W = \$1$ billion, at 13 hours, against a mean response of 50 hours. One bucket gives up little: on CCIP lanes the median of $\hat{E}(D)/E(D)$ is 1.13 at one hour and 1.05 at six hours.

Observation 7.4 (Several buckets)**.** *Token buckets in series, each supporting $\hat{E}$ at one of $D_1 < \cdots < D_K$, delay no legitimate history and let out $\min_k (b_k + r_k d)$ within d, which equals $\hat{E}(D_k)$ at each D_k.*

On Wormhole's combined traffic, four buckets matched to one hour, six hours, a day and a week stay within 6% of $\hat{E}$ and 19% of the floor E from one hour to 30 days, so an operator can approach Λ^* without an estimate of D.

7.1. The failed bridges' own traffic

We rebuilt the legitimate releases of failed bridges over the 150 days before each failure, from the events of their release contracts, and priced them in dollars (Appendix C): every failure with a recorded response of at least $20 million except Mixin, whose custody sat in hot wallets on many chains that we could not enumerate, and seven smaller ones whose release point we could read. The totals cover sixteen failures, which carried 89% of all the value; four more are shown apart: the 2024 Ronin failure met a deployed limit (Section 6), and Anyswap's routers, QBridge and the AFX bridge went into service 40, 59 and 71 days before their failures, too late for the fit below. Every release contract of a bridge gets its own limit, with its assets pooled in dollars, and a failure is charged the sum over the contracts in its scope (Corollary 4.2); for every failure here with several contracts, the taken keys or the flaw reached all of them. The floor, Λ^* with $h = 0$ fitted in sample, delays none of the 150 days, so it is the most a limit delaying none of that history could have guaranteed against a party pacing itself like the busiest legitimate traffic; it is a best case, not a design. The bucket is fitted out of sample, as a monthly refit would fit it: to the 90 days that end 30 days before the failure, for a six-hour response with a margin of 1.5 (Proposition 7.3), with its cost measured on the 30 days that follow, 115,811 transfers in all, and again one month earlier.

In sample. The floor keeps 71.4% of the value (78% averaged over failures; 49–90% over bootstrap samples of the failures and 66–85% leaving one out). Most of the loss is the 2022 Ronin failure, answered after six days: the busiest six days of Ronin's prior traffic released $855 million, more than the $540 million taken, $722 million of it in two withdrawals three days apart to wallets that public labels assign to one exchange. Without those two, Ronin's floor keeps 37% and the total 78.9%. On the other fifteen failures the floor keeps 85.4%.

Out of sample. A limit fitted to the past must delay users when traffic grows. The bucket keeps 72.2% (49–90%), but it delayed 17% of legitimate value in the month before the failures and 48% in the month before that, 0.8% and 12.7% of the transfers. About half of that delay is unavoidable: no defense that lets out as little within six hours could have delayed less than 8% and 23% (Proposition 7.2). Matched to one hour, a day or 50 hours instead of six, it keeps 72.7%, 69.5% and 67.7% and delays 16%, 18% and 20%: the cap binds for slow responses, so a short design time loses nothing there.

A review takes the transfers that a bucket fitted to the past cannot absorb. Sending transfers above the 99.9th percentile of the fit window's sizes to a review, and matching a bucket for the rest to one hour (Proposition 5.4), keeps 77.7% (53–96%), and that bucket delays only 2.4% and 1.0% of legitimate value. The reviewed transfers are few and large: a median of 1 per bridge and month and at most 148, carrying 38% and 48% of legitimate value. With reviews that take an hour, legitimate value waits 0.4 to 0.5 hours per dollar, and with a day, 9.1 to 11.7 hours; by hand they are slow, and the five withdrawals that Ronin's gateway has held for review since 2022 waited a median of 62 hours. A review protects only if the party cannot pass it, so it must check what the party cannot fake; Section 8 takes it to be the check that a deposit backs the release and asks what remains when that check fails.

Robustness. On Poly Network, 43% of the releases in 2021 and 31% in 2023 could not be priced, so its limits overstate what they keep. As governed, with the KEYS failures whose keys controlled the release contract left unseen, the bucket

TABLE 5. LIMITS FITTED TO EACH FAILED BRIDGE'S OWN TRAFFIC BEFORE THE FAILURE: SHARE OF THE VALUE RELEASED IN THE FAILURE THAT EACH WOULD HAVE KEPT AT THE RESPONSE TIME D. LEGIT: LEGITIMATE VALUE RELEASED PER DAY OVER THE 150 DAYS BEFORE THE FAILURE. FLOOR: THE LIMIT SHAPED LIKE ALL 150 DAYS, FITTED IN SAMPLE, WHICH DELAYS NONE OF THEM; IT IS THE MOST A LIMIT THAT DELAYS NO LEGITIMATE TRANSFER OF THAT HISTORY COULD GUARANTEE. BUCKET: A TOKEN BUCKET FITTED OUT OF SAMPLE TO THE 90 DAYS THAT END 30 DAYS BEFORE THE FAILURE, FOR $D = 6$ H, WITH A MARGIN OF 1.5. REVIEW: TRANSFERS ABOVE THE 99.9TH PERCENTILE OF SIZES IN THOSE 90 DAYS WAIT FOR A REVIEW THAT THE PARTY CANNOT PASS, AND A BUCKET FOR $D = 1$ H WITH A MARGIN OF 1.5 TAKES THE REST. ALARM: A PAUSE ONE HOUR AFTER THE FIRST RELEASE THAT NO DEPOSIT BACKS, WITH THE REVIEW DESIGN OR ALONE, WHERE THE CHECK FLAGS THE FAILURE AND SOMEONE OTHER THAN THE FAILING PARTY CAN STOP IT. A RANGE: THE VALUE RELEASED WITHIN THE FIRST HOUR IS KNOWN ONLY BETWEEN BOUNDS. EACH RELEASE CONTRACT IN A FAILURE'S SCOPE HAS ITS OWN LIMIT. NET: WEIGHTED BY THE VALUE NEVER RETURNED OR FROZEN. †LEFT OUT OF THE TOTALS (SECTION 7.1). TABLE 7 GIVES WHAT THE DESIGNS COST LEGITIMATE USERS AND KEEP WHEN THE CHECK FAILS.

						Kept at D				
						In sample	Out of sample		Alarm in 1 h	
Failure	Year	Class	$M	D (h)	Legit ($M/day)	Floor	Bucket	Review	Review	Alone
Poly Network	2021	check	610	2.8	14.06	83%	82%	92%	95%	0%
BSC Token Hub	2022	check	587	3.8	2.72	96%	94%	100%	100%	50%
Ronin 2022	2022	keys	540	145.9	24.27	0%	0%	0%	90%	0%
Wormhole	2022	check	322	1.1	5.98	65%	95%	96%	96%	0%
Liquid	2026	event	318	4.0	1.07	98%	99%	99%	100%	0%
Kelp rsETH	2026	check	294	0.8	3.12	95%	84%	93%	93%	0%
Nomad	2022	check	187	1.9	1.15	89%	93%	98%	99%	85%
Multichain 2023	2023	keys	129	5.3	1.53	86%	75%	81%	89%	15%
Harmony Horizon	2022	keys	100	8.8	5.08	12%	0%	35%	72%	1–2%
HECO bridge	2023	keys	86	2.6	0.38	90%	84%	90%	90%	0%
Orbit Bridge	2023	keys	82	1.2	0.31	98%	95%	97%	98%	0%
Qubit QBridge†	2022	event	80	3.6	0.01	99%	99%	100%	100%	0%
AFX bridge†	2026	keys	24	5.6	0.02	99%	99%	99%	99%	0%
Wanchain Cardano	2026	check	14	15.1	<0.01	100%	100%	100%	100%	0%
Ronin 2024†	2024	check	12	0.6	1.37	0%	0%	79%	79%	0%
Poly Network 2023	2023	keys	10	6.8	0.21	40%	15%	36%	55%	0–10%
Syscoin bridge	2026	check	9	11.1	<0.01	100%	100%	100%	100%	0%
Anyswap V3 router†	2021	keys	8	19.5	0.53	0%	87%	86%	97%	0%
Verus bridge July	2026	check	8	0.6	0.02	96%	97%	98%	98%	0%
Gravity Bridge	2026	event	5	11.0	<0.01	97%	100%	100%	100%	0%
By value (sixteen failures)						71%	72%	78%	94%	14%
90% bootstrap interval						49–90	49–90	53–96	92–97	
By failure						78%	76%	82%	92%	9%
Net						57%	57%	63%	92%	

keeps 69.2% and the review design 73.4%; with custody held by keys limited in the signers' software, 69.2% and 74.5%; on the ten failures whose first response was a pause, 67% and 71%; with response times five times longer, 67.2% and 69.9%; weighted by the value never returned or frozen, 57% and 63%. Fitted to 60 or 120 days, the review design keeps 77 to 78%.

7.2. Chainlink CCIP

Today's buckets. Of the 18 CCIP lanes that carried the most value from January to October 2026 (53.9% of it; Appendix C), 17 have an inbound bucket. Its release within D, divided by $\hat{E}(D)$ from the lane's own history, ranges from 0.17 to 8.09 at $D = 1$ h and from 0.34 to 13.94 at $D = 6$ h; it is above one, looser than the traffic needs, on 7 and 9 lanes. Every observed transfer passed the limits in force: a transfer above a bucket is rejected and leaves no record, one lane's traffic traces its outbound bucket, and on 5 of the 16 lanes of Table 6 (one lane has too short a history for it) more than half the value moved while the bucket was disabled. Replaying today's buckets therefore measures what these numbers would cost, not what users paid.

A schedule. Fitted to the first half of a lane's history, a bucket delays nothing in the second half on 8 of the 18 lanes, and up to 88% of the value where traffic grew. We therefore evaluated a schedule that every 30 days refits the bucket to the previous 90 days for a response time D and scales it by a margin μ (Table 6; transfers queue in order of arrival).

Matched to six hours with $\mu = 1.5$, the schedule delays 8.3% of legitimate value against 19.3% for today's buckets, but a few transfers larger than anything in the fit window wait a median of 27 hours; operators pass such transfers today by raising the limit by hand. Sending transfers above the 99th percentile of the fit window's sizes to a review and matching the bucket for the rest to one hour removes them: the bucket then delays 0.8% of the value, the release within one and six hours falls to 0.62 and 0.52 of today's on the median lane, and 2.4 transfers per lane and month, 24% of the value, wait for a review. This variant delays legitimate value less than today's buckets, 0.53 hours per dollar, if reviews take less than 2.2 hours. Five of the six review variants we tried release less than today's buckets at both response times, with break-even latencies of 1.6 to 2.5

TABLE 6. INBOUND BUCKETS AT THE RELEASE POINT OF 16 CCIP TOKEN LANES, REPLAYED ON EXECUTION TIMES IN 30-DAY SEGMENTS AGAINST TODAY'S BUCKETS. EACH 30 DAYS, A BUCKET IS REFITTED TO THE PREVIOUS 90 DAYS FOR A RESPONSE TIME D (IN HOURS) AND SCALED BY A MARGIN μ. REVIEW: TRANSFERS ABOVE THE 99TH PERCENTILE OF SIZES IN THE FIT WINDOW, ABOUT 2.4 PER LANE AND MONTH, WAIT FOR A REVIEW, AND THE BUCKET IS FITTED TO THE REST. DELAYED: SHARE OF VALUE THAT THE BUCKET DELAYED. DELAY: HOURS OF WAITING IN THE BUCKET PER DOLLAR OF TESTED VALUE. REVIEWED: SHARE OF VALUE SENT TO REVIEW. EVEN: THE REVIEW LATENCY, IN HOURS, AT WHICH THE DELAY PER DOLLAR, COUNTING A REVIEWED TRANSFER AS WAITING THAT LATENCY, EQUALS TODAY'S. 1 H, 6 H: RELEASE WITHIN THAT TIME, MEDIAN OVER LANES, RELATIVE TO TODAY'S BUCKET.

D	μ	Review	Delayed	Delay	Reviewed	Even	1 h	6 h
Today's buckets			19.3%	0.53	–	–	1	1
6	1.5	no	8.3%	10.76	–	–	1.57	0.60
1	1.5	no	9.7%	9.90	–	–	1.18	0.99
1	1	yes	4.9%	0.12	24%	1.7	0.42	0.35
1	1.5	yes	0.8%	0.01	24%	2.2	0.62	0.52
1	2	yes	0.0%	0.00	24%	2.2	0.83	0.70
6	1.5	yes	3.6%	0.34	24%	0.8	1.09	0.44
6	2	yes	0.7%	0.04	24%	2.1	1.46	0.58

hours.

7.3. Pooling across chains and assets

Most failures are confined: 36 of the 57 released value on one chain, and against them pooling helps only in series with per-chain caps (Corollary 4.2). Against a failure that reaches every chain it helps a lot: over twelve months, across the nine main chains of Wormhole's token bridge, caps sized to each chain let out 2.6 to 2.7 times what one pooled cap would, for windows from one hour to one day. Large transfers dominate short windows: without the 53 transfers above the Governor's size threshold, 5.8% of the value, the one-hour envelope of the combined traffic falls by a factor of 3.4, to $8.7 million. The Governor's daily caps seldom delay small transfers: over 24 months the trailing 24-hour outflow exceeded the cap in 415 hours on Sui and in at most 75 on any other chain. A pooled limit meters assets at weights in a common unit, and a weight that undervalues an asset by a factor x loosens the limit for it by that factor, which a cap on each asset bounds (Proposition A.3); the Governor, which takes the larger of a configured and the market price, undervalues only when its feed fails and the price has risen.

8. The response and the check

The shares so far take each failure's first public response as given, and for 30 of the 48 failures it is an acknowledgment. Where both an acknowledgment and a pause are recorded, the pause came first in 16 of 18, by a median of 2.2 hours, so an acknowledgment more likely overstates the time to the pause than understates it; 13 pauses are timed from the chain. We found no difference in response times between weekends and weekdays ($p = 0.75$) or across the hours of the day ($p = 0.40$), and they have not become faster: medians of 3.1 hours for the 19 failures of 2021 to 2023 and 5.0 hours for the 29 of 2024 to 2026.

TABLE 7. DESIGNS ON THE FAILED BRIDGES' TRAFFIC, FITTED A MONTH AHEAD: SHARE OF THE FAILURES' VALUE KEPT AT THE RESPONSE TIME WHEN THE CHECK THAT A DEPOSIT BACKS EACH RELEASE WORKS AND WHEN IT FAILS, AND THE COST TO LEGITIMATE USERS IN THE MONTH BEFORE EACH FAILURE AND THE MONTH BEFORE THAT (FIRST/SECOND NUMBER): VALUE AND TRANSFERS DELAYED (%), AND HOURS OF DELAY PER DOLLAR, A REVIEWED TRANSFER WAITING ONE HOUR. REVIEW: AS IN TABLE 5, THE REVIEW BEING THE CHECK. GATE: EVERY WITHDRAWAL HELD WHILE THE CHECK RUNS; BEHIND IT NOTHING, OR THE BUCKET OF TABLE 5, OR THAT BUCKET WITH A MARGIN OF 3. CUSTODY IN A CONTRACT THROUGHOUT; WITH CUSTODY AS BUILT, THE GATE KEEPS 96% WHEN THE CHECK WORKS.

	Kept (%), check		Cost		
Design	works	fails	Value	Transfers	h/$
Bucket	72	72	17/48	0.8/12.7	4.7/40.8
Review 1/1000	78	8	40/49	0.4/0.3	0.4/0.5
+ alarm 1 h	94	8	40/49	0.4/0.3	0.4/0.5
Gate 10 min	100	0	100	100	0.17
+ bucket	100	72	100	100	4.9/41.0
+ bucket, margin 3	100	64	100	100	1.6/9.8

A limit could change response times in both directions. A party paced by a limit withdraws at the pace of busy legitimate traffic, which hides the outflow through which 20 of the 57 failures, with 31% of the value, were found. If none of them had been answered, a bucket of $1 million per hour would still have contained 62.8% of the value, and the bucket fitted in Section 7.1, 61.0%; with response times five times as long, the former keeps 68.4%, and ten times as long, 62.8% against 25.4% for the 24-hour hold. Where no independent party could pause, setting $D = \infty$ leaves the bucket at 80.6%.

The check. The check we consider reads both chains from its own nodes, not through the verifiers, and compares for each asset what the release side has released or minted with what the sending side holds against it, as Liu et al.'s accounting does [12]. It flags a release that taken keys or a flawed check authorized without a deposit; a forged deposit event, since the event moves no value; and an unbacked mint when it is minted, before any redemption that a real burn backs. It flags every EVENT, KEYS and CHECK failure, 99.7% of the value, while a check that pairs each release with a deposit event misses the forged events and covers 89%. What the check flags, someone independent must still be able to stop. Where custody was held directly by keys that were taken, 8% of the value, no contract could hold or pause the withdrawals; like the limits of Section 7.1, the designs below assume custody in a contract (rule 1), and we report custody as built apart.

Three uses. Table 7 compares three uses of the check on the failed bridges' traffic. As a gate, it holds every withdrawal for 10 minutes while it runs, and by Theorem 7.1(iii) a failure it flags loses nothing: the gate keeps all of the value, 96% with custody as built, for 10 minutes, 0.17 hours per dollar, on every withdrawal. As the review of the largest transfers it spares 99.8% of them and keeps 78%. As an

alarm that pauses the bridge an hour after the first unbacked release, it adds little to the bucket, which keeps 73.0% because its burst lets most of a failure out within the hour, and lifts the review design to 94.5% (92–97%); alone, the alarm keeps 14%. Every use stands on the check. If it fails, the gate alone keeps nothing and the review design 8%, since the party's large withdrawals pass the review after its latency, while the bucket keeps 72% either way. A limit is thus the backstop for the check's failures, and its delay cost is the premium; a looser bucket, with a margin of 3, keeps 63.9% at 1.4 to 9.7 hours per dollar.

False pauses. From October 2024 to June 2025 on Wormhole, Liu et al.'s check raised 103 alerts, all from bugs in their code or the RPC services they used [12]; had each paused the bridge for an hour, the average legitimate dollar would have waited 0.009 hours more, far less than the gate costs. An alert must be re-checked before it pauses, and a pause by one party should lapse unless a second confirms it.

Is a short gate realistic? We ran the check per release in shadow on Wormhole's token bridge and Chainlink CCIP's token pools between Ethereum and BSC, reading both chains through public nodes. Behind each of their 13,261 recorded releases in the 42 days to 16 October 2026 (at least $29.5 million in all) it found a deposit that had moved the value, final before the release except for three CCIP transfers released three minutes after the deposit. It flagged thirteen releases of three tokens that pay a release in three parts; summing the parts clears them. Run live from 16 October by a monitor that was up 49% of the time, it checked 5,673 releases over 15.6 days: 2,904 as they happened and the rest on catching up after each outage. On the 2,846 it saw with no catch-up under way, it reached a verdict a median of 36 seconds after the release was included (99th percentile 164, at most 351); the other 58 arrived during a catch-up, and 30 of them took more than 10 minutes. It raised 22 alarms: a re-check cleared the 21 read errors from a public node, and the other was one more release paid in parts. It flagged all 8,701 test copies we made of live releases, citing no deposit or paying another amount or recipient. Bascule gates withdrawals on a reported deposit [13], and CCIP's risk network can halt lanes, by a flag now set by hand [14], [15]. On a chain with a fee market, keeping the pause out of blocks means outbidding it in every block; censorship by the chain's own block producers is outside our model.

9. What the stake must cover

A bridge whose verifiers post stake is commonly held safe when the stake exceeds what the bridge holds [1]. A limit at the release point changes what the stake must cover.

Corollary 9.1 (Stake per response)**.** *Let a limit Λ sit at the release point outside the verifiers' control, and let parties independent of the verifiers hold the pause and decide on confiscation. A coalition that controls the verifiers' attestations takes at most* $\min\{W, \beta_\Lambda(D)\}$ *before the response, and no limit of delay h lowers this below* $\min\{W, E((D-h)^+)\}$. *Corrupting the coalition is unprofitable if the stake it forfeits exceeds* $\sup_\tau \mathbb{E}[\min\{W, \beta_\Lambda(D_\tau)\}]$*, where D_τ is the response time to a failure that starts at τ. If the coalition controls the limit or the pause, its stake must exceed W; if it controls the confiscation, it forfeits nothing, and no stake deters it.*

The condition moves from the stock W to the flow over a response time, as STAKESURE does for validators that could reverse settled transactions [16] and HyperLoop for a sliding-window cap [17], here for any rule and any distribution of response times, if the stake cannot leave before the response [18]. It deters only the verifiers; in our KEYS failures outside parties took the keys, which a limit at the release point stops and stake does not.

Stakes today. We read the stakes behind the verifiers of nine systems on-chain (Table 8). Only the stake registered for Hyperlane on EigenLayer exceeds the value it secures, and only nominally: it is restaked for other services and cannot be confiscated yet. Of the rest, the highest ratio of stake to value held is 0.16, for Polygon's validators, whose $342 million, like Axelar's $16 million, can be confiscated only for other faults or not at all; Chainlink's LINK staking secures a price feed, not CCIP, and the arrangements that confiscate for an invalid attestation hold at most $1.3 million. The flow condition asks for much less: for a bridge as busy as Wormhole's nine chains, a limit that delays no legitimate transfer lets out $49.8 million on average before the response, which Polygon's validator stake would cover if it could be confiscated for an invalid attestation.

10. What operators can do

The results translate into nine rules of thumb.

1) *Put the limit where the funds are released, and keep its parameters and the pause out of the verifiers' hands.* Only there does it see KEYS failures and flawed receiving checks (Theorem 4.1); on our failures, a release-point bucket of $1 million per hour would have contained 87.2%, and no rule in the verifiers' software more than 27.0%.
2) *Make loosening slower than the response.* A role that can loosen a limit, or upgrade its contract, at once makes it absent for whoever holds that role. Delay both by more than the response time the operator can commit to, let a tightening take effect at once, and let the pauser cancel a pending change (Proposition 4.4).
3) *Hold every withdrawal for the latency of an independent check that value deposited on the sending chain backs it, and let the check pause.* The check must read both chains from its own nodes and compare value, not events; a failure it flags then loses nothing (Theorem 7.1(iii)), at 10 minutes per withdrawal on the failed bridges (Section 8). Give the pause to an independent party, re-check an alert before pausing, and let a pause by one party lapse unless a second confirms it.

4) *Keep a bucket behind the check, fitted to the bridge's own traffic.* It is what remains when the check fails, 72% here. Match it to the response time the operator can commit to, an hour to a few hours on these bridges rather than the mean, refit it monthly with a margin chosen on held-out data (Propositions 7.3 and 7.2; Table 6), and loosen it only as rule 2 allows. Several buckets in series follow the floor closely at every response time (Observation 7.4).
5) *If every withdrawal cannot wait, send only the largest transfers to the check, and never rely on a hold alone.* Reviewing the largest transfer in a thousand spares 99.8% of transfers and keeps 78%, but 8% if the check fails (Table 7); a hold without a check loses everything once the response misses it (Observation 5.5).
6) *Keep per-chain and per-asset caps under any pooled cap* (Corollary 4.2, Proposition A.3), and pool where the releases leave one contract.
7) *Net only inflows that cost the party.* A deposit that the bridge pays back on the other side adds that side's limit (Proposition 5.3); count it only if its payout there is held until a review.
8) *Keep one sender from exhausting a shared allowance.* Cycling funds through a shared bucket keeps it empty and delays everyone else; on the median CCIP lane this takes $3.2 million a day and capital for one round trip, and caps per sender raise it only as far as senders cannot multiply.
9) *Size the stake to what the limit lets out within a response time, with an independent confiscation decision* (Corollary 9.1).

11. Prototype

We implemented the rule as a Solidity contract at the release point: it holds the escrowed assets and is the only minter of the wrapped asset, so every release passes it. One token bucket meters all assets in a common unit at fixed weights. Each withdrawal is held for a configurable H, which may be zero, and then released by the bucket in order of arrival; a withdrawal larger than the bucket's capacity, which would block the queue, is rejected. An independent pauser can pause the contract and cancel queued withdrawals; the constructor rejects a pauser that is the bridge or the administrator, though code cannot check that the pauser is independent. Inflows do not refill the bucket, the parameters are fixed at deployment, an asset's weight can only rise after its listing, and anyone may send the execution call, so a stalled executor delays a release but cannot redirect it. A second contract adds the controls of Section 10: up to four buckets in series (Observation 7.4), a bucket per asset in the asset's own units (Corollary 4.2), and a tuner who can tighten the parameters at once but loosen them only after a delay, during which the pauser can cancel the change (Proposition 4.4). A queued withdrawal that a tightening has made larger than a bucket is dropped at the head of the queue, for the bridge to return or resubmit.

All 48 tests of the two contracts pass. Invariant tests (256 campaigns of 100 calls) check after every call that the releases between any two times $t_i < t_j$ stay within $b + r(t_j - t_i)$ and those within d of the first request within $b+r(d-H)$, and three seeded faults make them fail; a fuzz test of 1,000 runs checks the second contract against the minimum of its buckets' curves. Releasing one escrowed asset costs 38,135 gas without a limit; the first contract takes two transactions and 125,405 more gas, $0.06 per withdrawal at the median Ethereum fee of the prior 30 days (0.20 gwei, ether at $2,441), $0.19 at the 90th percentile and $9 at the 30 gwei common in 2021 and 2022. Four buckets with a per-asset bucket add 169,249 gas, $0.08 at the median fee.

End to end. We ran both contracts between two local chains on the 4,152 and 14,532 priced releases of Orbit Bridge and Nomad in the 150 days before their failures, in compressed time with destination blocks of 2 and 12 seconds. A relayer relays each deposit after 64 confirmations; a monitor run by an independent party matches every release request to a recorded deposit with the same amount and recipient and at least 64 confirmations, polls every 2, 12 or 60 seconds, and on a request without one pauses the contract and cancels the unmatched requests. It raised no alarm on 56,052 legitimate requests, each release requested at each of three limiters. In 636 injected failures, bursts of unbacked requests or streams paced below the bucket at points across the history, it detected each within one polling interval, and its pause was included at most 60 seconds after the first unbacked request, five 12-second blocks at 60-second polls; in 36 more with each party a separate process on the wall clock, after 2 to 60 seconds. Without a hold a burst still took up to the bucket's whole capacity, $2.6 million on Orbit Bridge and $13.1 million on Nomad, before the pause; with a 10-minute hold nothing left, for a 10-minute wait on every legitimate withdrawal (Theorem 7.1(iii)). In 12 more runs, each forging 20 deposits through a flawed path that records a deposit without moving value, a monitor that matched requests to recorded deposits raised no alarm while the bucket let out $2.9 million, and one that also required the deposit's transaction to move the amount into the escrow paused within 12 seconds; behind the hold nothing left. A monitor should keep checking a deposit until it is final: when reorganizations left 119 releases of a two-confirmation relayer unbacked, one that matched once caught at most 19, and one that kept checking caught all within 48 seconds.

12. Related work

Bridge failures. Augusto et al. systematize the designs of blockchain interoperability and the failures of bridges, and leave the effect of limits to future work [1]. Lee et al. classify bridge failures by the component that failed [19]; our entry stages are a coarser cut that decides what a limit sees. Zhang et al. survey 35 failures and recommend longer

withdrawal delays and alerts on large transfers, without sizing either [20]. Cao et al. measure flows by chain and day and recommend rate limits and circuit breakers [21]. SmartAxe finds missing checks in bridge contracts [22], and XScope, XChainWatcher and Liu et al.'s accounting check detect anomalies in bridge flows [12], [23], [24]; run as a gate, such a check is the bounded rule of Table 3, and run as an alarm it shortens the response (Section 8). Wu et al. collect 49 attacks and detect attack transactions by their call graphs, as BridgeShield does [9], [10], features that escape the floor of Theorem 7.1 as far as they separate attacks. ASAS-BridgeAMM adjusts withdrawal limits with other parameters under adversarial signals and replays traffic for solvency [25]. None of these evaluates limits against measured outflows and response times. Against Wu et al.'s and two other published lists, our dataset misses no unauthorized release of at least $1 million from a bridge (Appendix B).

Containment. Moore et al. model the containment of self-propagating code by reaction time, strategy and deployment [26], Williamson throttles hosts until a response [27], and circuit breakers in securities markets halt trading so that information can arrive [28]. Card networks cap the velocity of spending on a credential [29], and banks hold deposited checks until they clear [30], a gate on a check from outside the stream. Detecting a change in a stream with a bounded false-alarm rate is quickest change detection [31], [32]; Theorem 7.1(ii) is the case where the party can make the change undetectable.

Delays with a responder. Optimistic rollups and bridges release after a challenge period [33], payment channels wait out a dispute window that watchtowers watch [34], [35], and vaults hold a withdrawal while a recovery key can claw it back [36]. Each is a cancellable hold in our terms, a cliff whose loss follows the tail of the response time.

Limits and stake. The Wormhole Governor [8] and the ERC-7265 circuit breaker [37] are specified without a statement of what they contain, and releasing greedily maximizes the exits that caps on several windows allow [18]. XCLAIM collateralizes a vault against all the funds it holds [38], and Budish shows that the cost of trust scales with what an attack can take [39]; a limit with a bounded response caps that amount (Section 9).

Traffic shaping. Theorem 7.1 and Proposition 7.3 are the minimal arrival curve, the greedy shaper and the burstiness curve [4], [5], [11], [40], and Theorem 4.1 is the concatenation of shapers. Empirical and multi-rate envelopes [41], [42] correspond to our envelope and to buckets in series, adversarial queueing theory bounds injections by a rate over every window [43], and stochastic network calculus would replace the floor with one that legitimate traffic exceeds with a small probability [44]. What we add is the setting: failures that enter at different stages, limits the failing party may control, a response that cancels queued withdrawals, a check from outside the releases, and measured outflows and response times.

Response times. Zhou et al. time DeFi pauses [45], and Elem and Talmon find override times from half an hour for signer sets to days for governance votes [46].

13. Discussion

Limitations. The first public response stands for the pause (Section 8). The replays place one rule at a time, and more rules only tighten the bound (Theorem 4.1). The floor is an in-sample best case, the designs are fitted a month ahead on recorded traffic, and only the check has run beside live bridges, in shadow. The shares with the check assume that it works as Section 8 defines it and that an independent party stops what it flags; we treat its failure as all or nothing. Legitimate traffic is observed under the limits in force, so where a limit bound, the envelope understates demand (Section 7.2), and users who meet a limit may split their transfers or move to another bridge. The coding of entry stages rests on public sources, which the blind coder, a language model, read as we did for 55 of the 57 failures; a human recoding would test it further. The prototype has not been audited or connected to a deployed bridge.

Artifact. The anonymized artifact contains the data with a source for each measured field, the scripts that extract it and generate every number, table and figure, the exact checks of Sections 4–7, and the contracts, tests and testbed.

Ethics. Every input to this paper is public, and we sent no transaction to a live contract: the replays are computations, the prototype ran on local chains, and the shadow check only read public nodes. We describe failures as mechanisms, without addresses or transaction sequences, and notified the maintainers of Hyperlane and of Wormhole's NTT and Governor of what Section 5 finds about their netting rules in October 2026, before submission.

AI disclosure. Generative AI tools, including Codex and Claude, assisted with literature searches, coding assistance and prose polishing. The authors conceived the paper's core ideas, developed its methods, verified the correctness and originality of all content, including the references, and take full responsibility for it.

14. Conclusion

A withdrawal limit cannot tell an unauthorized withdrawal from a legitimate one, so it buys time for a response, and a bridge's own traffic sets a floor that no defense seeing only amounts and times can beat. On the largest failures since 2021, no limit that delayed none of a bridge's prior traffic could have guaranteed more than 71% of the value at the response times operators achieved, and a bucket fitted a month ahead keeps 72% while delaying 17% to 48% of legitimate value. Used as a short gate in front of the bucket, a check that a deposit backs each release would have stopped every failure it flags where a contract held the funds; the bucket is what remains when the check fails. The time a limit buys is worth what the response does with it.

References

[1] A. Augusto, R. Belchior, M. Correia, A. Vasconcelos, L. Zhang, and T. Hardjono, "SoK: Security and privacy of blockchain interoperability," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024, pp. 3840–3865.

[2] ——, "SoK: Security and privacy of blockchain interoperability (extended version)," https://www.dpss.inesc-id.pt/~mpc/pubs/augusto2024.pdf, 2024.

[3] CertiK, "Axie infinity – audit," https://web.archive.org/web/20240326005747/https://docs.roninchain.com/assets/files/CertiK-Audit-for-Axie-Infinity---Audit-v8-1bfcb82b195442bf34a28ed2fdbde6c5.pdf, 2022, ronin bridge v2, verified June 21, 2022.

[4] R. L. Cruz, "A calculus for network delay, part I: Network elements in isolation," *IEEE Transactions on Information Theory*, vol. 37, no. 1, pp. 114–131, 1991.

[5] J.-Y. Le Boudec and P. Thiran, *Network Calculus: A Theory of Deterministic Queuing Systems for the Internet*, ser. Lecture Notes in Computer Science. Springer, 2001, vol. 2050.

[6] Chainlink, "CCIP token lane configuration," File lanes.json, https://github.com/smartcontractkit/documentation/tree/main/src/config/data/ccip/v1_2_0/mainnet.

[7] DefiLlama, "Hacks," https://defillama.com/hacks.

[8] Wormhole Foundation, "Governor," Wormhole whitepaper 0007, https://github.com/wormhole-foundation/wormhole/blob/ff186e441aff870bfee0481766934fbc3961d7bc/whitepapers/0007_governor.md.

[9] J. Wu, K. Lin, D. Lin, B. Zhang, Z. Wu, and J. Su, "Safeguarding blockchain ecosystem: Understanding and detecting attack transactions on cross-chain bridges," in *Proceedings of the ACM Web Conference 2025 (WWW '25)*. ACM, 2025, pp. 4902–4912.

[10] D. Lin, S. Lu, Z. Liu, J. Wu, J. Fang, J. Su, B. Song, Q. Xia, and Z. Zheng, "BridgeShield: Risk-aware graph modeling for cross-chain bridge attack detection," arXiv:2508.20517v2, 2026.

[11] S. H. Low and P. P. Varaiya, "Burstiness bounds for some burst reducing servers," in *Proceedings of IEEE INFOCOM '93, the Conference on Computer Communications*. San Francisco, CA, USA: IEEE Computer Society Press, 1993, pp. 2–9.

[12] E. Liu, E. Luo, J. C. Yan, K. Izhikevich, S. Grant, D. Stefan, G. M. Voelker, and S. Savage, "Count of Monte Crypto: Accounting-based defenses for cross-chain token bridges," arXiv:2410.01107v3, 2026.

[13] Cubist, "Introducing the bascule drawbridge for Bitcoin bridge security," Cubist blog, https://cubist.dev/blog/introducing-the-bascule-drawbridge-for-bitcoin-bridge-security, 2024.

[14] Chainlink, "Chainlink CCIP's defense-in-depth security and the risk management network," Chainlink blog, https://chain.link/blog/ccip-risk-management-network, Oct. 2023.

[15] ——, "CCIP architecture overview," Chainlink Documentation, https://docs.chain.link/ccip/concepts/architecture/overview, 2026, CCIP 2.0.

[16] S. Deb, R. Raynor, and S. Kannan, "STAKESURE: Proof of stake mechanisms with strong cryptoeconomic safety," arXiv:2401.05797, 2024.

[17] A. Kate, E. V. Mangipudi, C. Nomula, R. Ramesh, A. Terzoglou, and J. Tobkin, "HyperLoop: Rationally secure efficient cross-chain bridge," Cryptology ePrint Archive, Paper 2025/176, 2025.

[18] M. Neuder, M. M. Pai, and M. Resnick, "Optimizing exit queues for proof-of-stake blockchains: A mechanism design approach," in *6th Conference on Advances in Financial Technologies (AFT 2024)*, ser. LIPIcs, vol. 316, 2024, pp. 20:1–20:22.

[19] S.-S. Lee, A. Murashkin, M. Derka, and J. Gorzny, "SoK: Not quite water under the bridge: Review of cross-chain bridge hacks," in *2023 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. Dubai, United Arab Emirates: IEEE, 2023, pp. 1–14.

[20] M. Zhang, X. Zhang, Y. Zhang, and Z. Lin, "Cross-chain bridges: Attack taxonomy, defenses, and open problems," in *The 27th International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2024)*. Padua, Italy: ACM, 2024, pp. 298–316.

[21] Y. Cao, M. Zheng, L. W. Cong, S. Li, and X. Wang, "The price of interoperability: Exploring cross-chain bridges and their economic consequences," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 10, no. 2, 2026, article 52.

[22] Z. Liao, Y. Nan, H. Liang, S. Hao, J. Zhai, J. Wu, and Z. Zheng, "SmartAxe: Detecting cross-chain vulnerabilities in bridge smart contracts via fine-grained static analysis," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 249–270, 2024.

[23] J. Zhang, J. Gao, Y. Li, Z. Chen, Z. Guan, and Z. Chen, "Xscope: Hunting for cross-chain bridge attacks," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE '22)*. Rochester, MI, USA: ACM, 2022, pp. 1–4.

[24] A. Augusto, R. Belchior, J. Pfannschmidt, A. Vasconcelos, and M. Correia, "XChainWatcher: Identifying anomalies in cross-chain bridges," in *Proceedings of the 26th International Middleware Conference (Middleware '25)*. ACM, 2025, pp. 413–426.

[25] S. You, A. Joshi, A. Kuehlkamp, and J. Nabrzyski, "ASAS-BridgeAMM: Trust-minimized cross-chain bridge AMM with failure containment," arXiv:2601.12434, 2026.

[26] D. Moore, C. Shannon, G. M. Voelker, and S. Savage, "Internet quarantine: Requirements for containing self-propagating code," in *Proceedings of IEEE INFOCOM 2003, the 22nd Annual Joint Conference of the IEEE Computer and Communications Societies*, vol. 3. San Francisco, CA, USA: IEEE, 2003, pp. 1901–1910.

[27] M. M. Williamson, "Throttling viruses: Restricting propagation to defeat malicious mobile code," in *Proceedings of the 18th Annual Computer Security Applications Conference (ACSAC 2002)*. Las Vegas, NV, USA: IEEE Computer Society, 2002, pp. 61–68.

[28] B. C. Greenwald and J. C. Stein, "Transactional risk, market crashes, and the role of circuit breakers," *The Journal of Business*, vol. 64, no. 4, pp. 443–462, Oct. 1991.

[29] R. J. Bolton and D. J. Hand, "Statistical fraud detection: A review," *Statistical Science*, vol. 17, no. 3, pp. 235–255, 2002.

[30] Board of Governors of the Federal Reserve System, "Regulation CC: Availability of funds and collection of checks," 12 CFR Part 229, 2026.

[31] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.

[32] A. Tartakovsky, I. Nikiforov, and M. Basseville, *Sequential Analysis: Hypothesis Testing and Changepoint Detection*. Boca Raton, FL, USA: CRC Press, 2014.

[33] H. Kalodner, S. Goldfeder, X. Chen, S. M. Weinberg, and E. W. Felten, "Arbitrum: Scalable, private smart contracts," in *27th USENIX Security Symposium (USENIX Security 18)*. Baltimore, MD, USA: USENIX Association, 2018, pp. 1353–1370. [Online]. Available: https://www.usenix.org/conference/usenixsecurity18/presentation/kalodner

[34] P. McCorry, S. Bakshi, I. Bentov, S. Meiklejohn, and A. Miller, "Pisa: Arbitration outsourcing for state channels," in *Proceedings of the 1st ACM Conference on Advances in Financial Technologies (AFT '19)*. Zurich, Switzerland: ACM, 2019, pp. 16–30.

[35] Z. Avarikioti, O. S. T. Litos, and R. Wattenhofer, "Cerberus channels: Incentivizing watchtowers for Bitcoin," in *Financial Cryptography and Data Security (FC 2020)*, ser. Lecture Notes in Computer Science, J. Bonneau and N. Heninger, Eds., vol. 12059. Cham: Springer, 2020, pp. 346–366.

[36] M. Möser, I. Eyal, and E. G. Sirer, "Bitcoin covenants," in *Financial Cryptography and Data Security: FC 2016 International Workshops, BITCOIN, VOTING, and WAHC*, ser. Lecture Notes in Computer Science, J. Clark, S. Meiklejohn, P. Y. A. Ryan, D. Wallach, M. Brenner, and K. Rohloff, Eds., vol. 9604. Berlin, Heidelberg: Springer, 2016, pp. 126–141.

[37] D. Campos, B. Dimovski, P. Dumonet, and M. Bank, "ERC-7265: Circuit breaker," Draft, https://github.com/DeFi-Circuit-Breaker/EIPs/blob/master/EIPS/eip-7265.md, 2023.

[38] A. Zamyatin, D. Harz, J. Lind, P. Panayiotou, A. Gervais, and W. Knottenbelt, "XCLAIM: Trustless, interoperable, cryptocurrency-backed assets," in *2019 IEEE Symposium on Security and Privacy (SP)*. San Francisco, CA, USA: IEEE, 2019, pp. 193–210.

[39] E. Budish, "Trust at scale: The economic limits of cryptocurrencies and blockchains," *The Quarterly Journal of Economics*, vol. 140, no. 1, pp. 1–62, Feb. 2025.

[40] C.-S. Chang, *Performance Guarantees in Communication Networks*, ser. Telecommunication Networks and Computer Systems. London: Springer, 2000.

[41] D. E. Wrege, E. W. Knightly, H. Zhang, and J. Liebeherr, "Deterministic delay bounds for VBR video in packet-switching networks: Fundamental limits and practical trade-offs," *IEEE/ACM Transactions on Networking*, vol. 4, no. 3, pp. 352–362, 1996.

[42] E. W. Knightly and H. Zhang, "D-BIND: An accurate traffic model for providing QoS guarantees to VBR traffic," *IEEE/ACM Transactions on Networking*, vol. 5, no. 2, pp. 219–231, 1997.

[43] A. Borodin, J. Kleinberg, P. Raghavan, M. Sudan, and D. P. Williamson, "Adversarial queuing theory," *Journal of the ACM*, vol. 48, no. 1, pp. 13–38, 2001.

[44] M. Fidler and A. Rizk, "A guide to the stochastic network calculus," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 1, pp. 92–105, 2015.

[45] L. Zhou, X. Xiong, J. Ernstberger, S. Chaliasos, Z. Wang, Y. Wang, K. Qin, R. Wattenhofer, D. Song, and A. Gervais, "SoK: Decentralized finance (DeFi) attacks," in *2023 IEEE Symposium on Security and Privacy (SP)*, 2023, pp. 2444–2461.

[46] O. Elem and N. Talmon, "Legitimate overrides in decentralized protocols," arXiv:2602.12260, 2026.

[47] Chainlink, "CCIP API, version 2," https://api.ccip.chain.link/v2/messages.

[48] DefiLlama, "Coins API," https://api-docs.defillama.com/.

[49] Wormhole Foundation, "Wormholescan API," https://api.wormholescan.io/api/v1/operations.

TABLE 8. STAKE BEHIND THE VERIFIERS AGAINST THE VALUE HELD AT ONE TIME (APPENDIX C), IN MILLIONS OF US DOLLARS. STAKE IS READ ON-CHAIN. HELD: ASSETS IN THE ESCROW AND LOCK CONTRACTS THAT THE VERIFIERS' ATTESTATIONS RELEASE, READ ON-CHAIN FOR POLYGON (IN PART), ACROSS AND CCIP (THE ETHEREUM LOCK-RELEASE POOLS), AND OTHERWISE THE TOTAL REPORTED BY DEFILLAMA FOR HYPERLANE, LOMBARD AND LAYERZERO, AXELARSCAN FOR AXELAR AND WORMHOLESCAN FOR WORMHOLE. CONFISCABLE: WHETHER PUBLISHED RULES LET STAKE BE TAKEN FOR ATTESTING TO AN INVALID MESSAGE. NOMINAL: THE STAKE REGISTERED FOR THE SERVICE, WHICH ITS OPERATORS ALSO RESTAKE FOR OTHER SERVICES, AT 1 ETH PER SHARE. CHAINLINK'S LINK STAKING SECURES A PRICE FEED AND IS NOT COUNTED FOR CCIP.

System	Stake	Confiscable	Held	Ratio
Polygon PoS validators	342.4	no: slashing removed	2,184	0.16
Axelar validators	16.0	no: double signing, downtime	183.5	0.087
Hyperlane HYPER vault (Symbiotic)	1.32	yes, with a council	76.5	0.017
Hyperlane EigenLayer service	3,490 nominal	no: not enabled	76.5	n/a
Lombard vault for LBTC on CCIP	0.144	no published conditions	632.1	0.0002
Across proposer bond	0.0011	if a dispute succeeds	17.85	0.00006
LayerZero verifiers	0	none found on-chain	11,110	0
Chainlink CCIP	0	no: staking covers a feed	1,454	0
Wormhole guardians	0	no stake	1,648	0

Appendix A. Proofs

We also checked these statements by exhaustive exact computation on small discrete instances, except Observation 5.5, Proposition 7.2, Corollaries 9.1 and A.1, the random detector of Theorem 7.1(ii), the hold of its part (iii) and the cap per message in Table 3; the scripts are part of the artifact.

A.1. Theorem 4.1, Corollary 4.2 and Observation 4.3

Measure time from τ. For (iii), let R_0 be the cumulative value that the party submits on the route, so $R_0(0) = 0$ and $R_0 \le W$, and let R_j be the cumulative value that the j-th active limit has released. A limit that lets out at most $\beta_j(u)$ within any interval of length u cannot release more than it has received, so for $s \le t$

$$R_j(t) \le R_j(s) + \beta_j(t-s) \le R_{j-1}(s) + \beta_j(t-s), \quad \text{(A.1)}$$

that is, $R_j \le R_{j-1} \otimes \beta_j$. A hold releases nothing it received less than H earlier, which is (A.1) with $\beta_j = \delta_H$. A rule that also releases its queue after a time A satisfies (A.1) with the curve $\max\{\beta_j(u), W\,\mathbf{1}[u > A]\}$: for $t - s \le A$, whatever the timer releases in $(s, t]$ was submitted by s and still queued at s. The other rules of Table 1 let out at most their curve within any interval, because the quiet state is their most permissive state. Since $\otimes$ is associative, $R_k \le R_0 \otimes \beta_1 \otimes \cdots \otimes \beta_k$, and taking $s = d$ and $s = 0$ in the infimum gives $(R_0 \otimes g)(d) \le \min\{W, g(d)\}$ for a curve g with $g(0) = 0$. A greedy shaper whose curve σ is subadditive with $\sigma(0) = 0$ turns an input R into exactly $R \otimes \sigma$ [5, Sec. 1.5], and a hold turns it into $R \otimes \delta_H$; token buckets and sliding windows are such shapers, so submitting W at τ attains the first bound. For the second, $f \otimes g \le f(d) + g(0)$; if f and g are concave with $f(0) = g(0) = 0$, then $s \mapsto f(s) + g(d - s)$ is concave on $[0, d]$ and takes its minimum at an end point. For (ii), every unit released on a route in $\mathcal{R}$ passes some limit $j \in \mathcal{C}$, and limit j releases at most $\beta_j(d)$ within d, whatever its input; summing over $\mathcal{C}$ gives the bound. For (i), the route without an active limit lets the party take W at once. Corollary 4.2 applies (ii) twice, with $\mathcal{C} = \{\Sigma\}$ and with $\mathcal{C} = K$. Observation 4.3 is (i): an unauthorized mint enters at the receiving chain, and the escrow limit of the sending chain lies on no route the minted claims take unless they are redeemed.

A.2. Proposition 4.4

Before T no loosening announced at or after τ has taken effect, and a tightening only lowers the curve, so the limit lets out at most $\beta(d)$ within $d \le T$. At T the party can lift the limit and take what remains, so the curve is W for

TABLE 9. THE 20 LARGEST FAILURES, 96% OF THE VALUE. CLASS: STAGE WHERE THE WITHDRAWALS ENTERED. OUTFLOW: FIRST TO LAST UNAUTHORIZED RELEASE. RESPONSE: FIRST PUBLIC RESPONSE (A: ACKNOWLEDGMENT, P: PAUSE). FOUND: HOW THE OPERATOR LEARNED (ALERT: A THIRD PARTY'S PUBLIC ALERT; OPERATOR: OWN MONITORING; USER: A USER'S REPORT OR FAILED WITHDRAWAL; SHORTFALL: A SHORTFALL IN THE ESCROW). PAUSE: WHO COULD PAUSE (INDEP.: A PARTY INDEPENDENT OF THE FAILURE; PARTY: THE FAILING PARTY; NONE: NO PAUSE ON THE PATH). RECOVERED: RETURNED OR FROZEN AFTERWARDS. ALL 57 FAILURES, WITH SOURCES, ARE IN THE ARTIFACT.

Failure	Year	$M	Class	Side	Outflow	Response	Found	Pause	Recovered
Poly Network	2021	610	check	escrow	36 min	2 h 47 min a	–	indep.	610
BSC Token Hub	2022	587	check	mint	2 h 16 min	3 h 46 min p	–	indep.	436
Ronin 2022	2022	540	keys	escrow	1 min	6 d 1 h p	user	indep.	70
Wormhole	2022	322	check	both	7 min	1 h 8 min p	operator	indep.	140
Liquid	2026	318	event	both	0 min	3 h 57 min p	operator	indep.	268
Kelp rsETH	2026	294	check	escrow	0 min	45 min p	operator	indep.	71
Nomad	2022	187	check	escrow	2 h 33 min	1 h 52 min a	alert	indep.	39
Mixin	2023	143	keys	escrow	1 h 25 min	2 d 3 h a	–	none	0
Multichain 2023	2023	129	keys	escrow	1 h 55 min	5 h 16 min a	alert	none	66
Harmony Horizon	2022	100	keys	escrow	19 min	8 h 46 min p	operator	party	0
HECO bridge	2023	86	keys	escrow	20 min	2 h 35 min a	alert	–	0
Orbit Bridge	2023	82	keys	escrow	17 min	1 h 13 min p	operator	indep.	0
Qubit QBridge	2022	80	event	mint	14 min	3 h 37 min a	alert	indep.	0
AFX bridge	2026	24	keys	escrow	0 min	5 h 34 min a	alert	–	0
Wanchain Cardano	2026	14	check	escrow	8 min	15 h 4 min a	alert	indep.	0
pNetwork pBTC	2021	13	check	escrow	6 min	3 h 3 min a	operator	indep.	0
Ronin 2024	2024	12	check	escrow	34 min	38 min p	alert	indep.	12
Verus bridge May	2026	12	check	escrow	0 min	–	alert	indep.	8
Garden solver	2025	11	keys	other	4 min	5 h 1 min a	alert	none	0
THORChain 2026	2026	11	keys	escrow	3 h 28 min	52 min p	operator	none	0

TABLE 10. SHARE OF THE VALUE OF THE FAILURES THAT EACH RULE WOULD HAVE CONTAINED, UNDER OTHER READINGS OF THE RESPONSE TIME AND OTHER WEIGHTINGS. BASE: AS IN SECTION 5. 90% INTERVAL: 5TH TO 95TH PERCENTILE OF THE BASE SHARE OVER 2,000 RESAMPLES OF THE FAILURES. $2D$, $D + 1$ H, $D + 6$ H: THE FIRST RESPONSE DOUBLED OR DELAYED. PAUSES: ONLY THE 18 FAILURES WHOSE FIRST RESPONSE WAS A PAUSE. BY EVENT: EACH FAILURE WEIGHTED EQUALLY. WITHOUT LARGEST: THE LARGEST FAILURE, 17% OF THE VALUE, LEFT OUT; WITHOUT 3 LARGEST: THE THREE LARGEST, 47%. MATCHED: ONE LIMIT AT THE RELEASE POINT THAT DELAYS NO TRANSFER OF THE COMBINED TRAFFIC OF WORMHOLE'S NINE CHAINS OVER THE LAST 12 OR 24 MONTHS, APPLIED TO EVERY FAILURE; WITH REVIEW, TRANSFERS ABOVE THE GOVERNOR'S SIZE THRESHOLD WAIT FOR A REVIEW AND THE LIMIT IS MATCHED TO THE REST. ALL RULES BUT THE LAST SIT AT THE RELEASE POINT.

Rule	Base	90% interval	$2D$	$D + 1$ h	$D + 6$ h	Pauses	By event	Without largest	Without 3 largest
Bucket, $1M per hour	87.2%	78 to 93%	80.3%	86.5%	83.3%	89.7%	32.0%	85.0%	84.1%
Hold, 24 hours	80.7%	57 to 99%	79.8%	80.7%	80.4%	76.2%	79.2%	76.9%	91.4%
Cumulative cap, $10M	90.1%	84 to 93%	90.1%	90.1%	90.1%	92.9%	27.4%	89.1%	83.8%
Matched, 12 months	75.3%	63 to 81%	71.7%	75.0%	73.7%	80.8%	19.0%	73.4%	64.5%
Matched, 24 months	32.1%	14 to 47%	30.3%	30.7%	29.5%	36.7%	6.0%	30.2%	23.6%
Matched with review, 12 months	85.4%	76 to 90%	80.1%	84.7%	81.5%	89.0%	24.9%	83.5%	78.8%
Matched with review, 24 months	58.6%	37 to 75%	50.2%	55.5%	41.2%	58.7%	14.9%	54.1%	57.7%
Bucket in the verifiers' software	25.5%	7 to 49%	25.0%	25.3%	24.2%	36.7%	10.7%	30.6%	48.5%

$d > T$. If $D \le T$, the response cancels the pending change before it takes effect.

A.3. Proposition 5.1 and Observation 5.2

For each family the bound follows from the rule's definition, and the strategy that attains it is as follows. A cap per message releases C for each of the $N(d)$ messages processed. A sliding window releases at most C in any window of length w, hence $C\lceil d/w \rceil$ within d, attained by submitting C at the start of each window. A calendar window allows C in the window that contains τ and C in each window that starts within d; starting just before a boundary gives $C(\lceil d/w \rceil + 1)$. A token bucket starts full and refills at the rate r. A hold releases a withdrawal only after H and the response cancels what is still held, so nothing leaves if $D \le H$, and W submitted at τ leaves at $\tau + H$ otherwise. A hold followed by a bucket has the curve $\delta_H \otimes (b + r\,\cdot)$ by Theorem 4.1. Under an automatic release after A, the rule underneath releases its curve and the queue leaves at $\tau + A$ without counting. A share p of the balance per window lets at most p of what remains leave in each window, so at most $W(1 - (1 - p)^n)$ has left after n windows. The two net-flow rows are Proposition 5.3, and a cumulative cap releases at most θ until a review resets it. For Observation 5.2: after any interval of length T without releases the rule allows a positive release, rT for a bucket, C once a window has slid or reset, the queue when a timer fires, and pX for a share of the balance X that remains. The party takes that allowance at the end of each such interval; the first three restore at least some $\varepsilon > 0$ each time, and the balance left after k windows of the last is at most $(1-p)^k W$, so the take tends to W.

A.4. Propositions 5.3 and 5.4, Observation 5.5

For (i), let the party release y on side A within d and count an inflow I, so that $y \le \beta_A(d) + I$; its net take is

$y - cI \le \beta_A(d) + (1-c)I$, and it cannot exceed W. For (ii), let P be what side B pays back of the deposits, so $P \le \beta_B(d)$. The net take is $y - I + P \le \beta_A(d) + \beta_B(d)$. To reach it, the party releases what A allows, deposits it back on A, which restores A's allowance at once, and collects it on B as fast as B's limit allows; it needs no capital of its own, and only the deposits not yet paid out at the response are lost to it. For Proposition 5.4, (i): the party keeps every withdrawal outside $\mathcal{K}$, for instance by splitting below s, and all of them go to Λ, whose curve they attain. (ii) is Theorem 7.1 applied to the traffic outside $\mathcal{K}$, which is all that Λ must pass. (iii) is the automatic-release row of Table 3. Observation 5.5 is the linearity of expectation.

A.5. Theorem 7.1, Propositions 7.3 and A.2, Observation 7.4, and two further results

Corollary A.1 (Window length)**.** *A sliding window of length w with cap $E(w)$ delays no legitimate history and lets out $E(w)$ within any $d \le w$, where the floor is $E(d)$.*

Proposition A.2 (A bucket for uncertain response times)**.** *Let D have distribution G with mean $\mu < \infty$. Every bucket with delay zero lets out on average $\mathbb{E}_G[b + rD] \ge \hat{E}(\mu)$ before the response, with equality for the bucket that supports $\hat{E}$ at μ.*

Proposition A.3 (Value weighting)**.** *Let a limit meter asset k at a weight ω_k in a common unit while its unit value is v_k, and let it let out $\beta(d)$ common units within d. The loss is at most $\min\{W, \beta(d)\max_k v_k/\omega_k\}$, attained when enough of the asset with the largest ratio is available. A cap of κ_k units on each asset adds the bound $\sum_k v_k\kappa_k$.*

For the bound in Theorem 7.1, fix $u = (d-h)^+$ and $\varepsilon > 0$, and pick a history $\ell \in \mathcal{T}$ and a time s with $\ell(s+u) - \ell(s) > E(u) - \varepsilon$. Because $\mathcal{T}$ is closed under shifts and under dropping a prefix, the part of ℓ from s on, moved to start at τ, is again a legitimate history, and the party submits it. The limit cannot tell it from legitimate traffic and releases each withdrawal within h, so by $\tau + h + u \le \tau + d$ it has released more than $E(u) - \varepsilon$. The same argument gives (ii): a defense whose decisions depend only on the amounts and times of the withdrawals it receives and of its own releases decides on the replay as it would on a legitimate history from its quiet state, so it releases each withdrawal within h and does not pause. A randomized one pauses the replay within d with probability at most α and otherwise releases more than $E(u) - \varepsilon$ by $\tau + d$, so its expected loss exceeds $(1-\alpha)(E(u) - \varepsilon)$ for every $\varepsilon > 0$. For (iii), every unauthorized withdrawal is unbacked and no legitimate one is, so the alarm pauses no legitimate history and pauses by $\tau + \ell$ after a failure; nothing is released after the pause, and until then Λ releases at most $\beta_\Lambda(\ell)$. With a hold $H \ge \ell$ in front, nothing submitted after τ leaves before $\tau + H \ge \tau + \ell$, by which time the pause has cancelled it. If the check passes the failure's withdrawals, the hold and Λ in series let out at most $(\delta_H \otimes \beta_\Lambda)(d) = \beta_\Lambda((d-H)^+)$ by Theorem 4.1. For Proposition 7.2, we fix disjoint intervals $I_k = [s_k, s_k + d - h)$. A withdrawal arriving in I_k that waits at most h leaves in $[s_k, s_k + d)$, within which the defense releases at most x, so at least $(\ell(I_k) - x)^+$ of what arrives in I_k waits more than h; summing over the disjoint I_k gives $F(x)$. For Λ^*, the envelope is subadditive, so a legitimate history delayed by h already meets the curve E and the greedy shaper adds no delay [4], [5]; the party's withdrawals reach the shaper at $\tau + h$ at the earliest. For Corollary A.1, no legitimate window of length up to w carries more than $E(w)$, and the party attains $E(w)$ by submitting it at once. For Proposition 7.3, a bucket is a greedy shaper with the curve $b + ru$, so it delays no legitimate history exactly when that line lies above E, that is, above $\hat{E}$, and $\hat{E}$ is the lower envelope of these lines. Proposition A.2 follows: $\mathbb{E}_G[b + rD] = b + r\mu \ge \hat{E}(\mu)$ for every such line, with equality for a line that supports $\hat{E}$ at μ. Observation 7.4 is Theorem 4.1(iii) for concave curves. For Proposition A.3, a common unit is worth v_k/ω_k when spent on asset k, so the best use of $\beta(d)$ units takes the assets in decreasing order of that ratio, and caps on the assets bound the loss by $\sum_k v_k\kappa_k$.

A.6. Corollary 9.1

By Theorem 4.1, the coalition's withdrawals pass Λ, which it cannot switch off, so it takes at most $\min\{W, \beta_\Lambda(D)\}$ before the independent pause cancels the queue, and Theorem 7.1 gives the floor. The coalition chooses its start τ, so the condition takes the supremum of the expectation over τ. If the coalition controls the limit, the limit counts as absent; if it controls the pause, $D = \infty$ and the take reaches W by Observation 5.2; if it controls the confiscation, it forfeits nothing.

Appendix B. Failure data

Universe. We started from the records of DefiLlama's list of incidents [7] of at least $1 million from 2021 to 2026 that it flags as bridge incidents or classes as cross-chain, added the incidents of the 2024 systematization's tables that it does not flag [1], 69 in all, and kept the 57 in which an unauthorized party released value from a bridge's escrow, minting or liquidity. We left out withdrawals that the owners had authorized, such as drains of users' token approvals, events outside bridges, duplicate records and an operator's own operations. We checked coverage against three published lists: Wu et al.'s 49 attacks [9], Zhang et al.'s 35 [20] and Lee et al.'s eight cases [19]. Of their 92 entries, 43 are among our failures and 9 among the records we left out; of the rest, 32 released less than $1 million or nothing, seven released from no bridge (a front end, users' approvals, a token contract or a protocol's own treasury), and one describes an attack that never happened. Of our failures, 33 postdate all three lists.

Coding. For each failure we recorded the value released, the class, the side that released, the times of the first and the last unauthorized release, the time by which 90% of

the value had left, the first public alert by a third party, the operator's first acknowledgment, the pause, the limits in force, the chains and assets drained, how the operator learned of the failure, who could pause, whether the taken keys also controlled the releasing contract, whether a flaw also reached a limit there, and the value later returned or frozen, with a source for each measured value. The class follows the entry stage: EVENT when the sending chain recorded a deposit that moved no value and the release side honored it; KEYS when keys that authorize releases were under outside control, so releases carried valid authorizations; CHECK when the receiving check, or an off-chain verifier that authorizes the release, accepted a message that matched no deposit with no key taken; OTHER otherwise. A key obtained through a software flaw counts as KEYS. A second pass recoded every failure from the post-mortems; it disagreed on three failures, and the EVENT share is 11.1% under the first coding and 11.1% under the second. A third coder, a separate language-model session, then coded every failure blind, from a codebook, our descriptions of each failure and the public sources, with the earlier codes removed from the input: class, the check's location, every release leg and whether it passed the verifiers, whether a contract held the custody, who could pause, and the first response. That coding agreed on 55 classes ($\kappa = 0.95$) and 54 pause holders; its legs put 27% of the value through the verifiers, and its reading of the custody gives the 8 failures held by keys in Section 4. We recoded Orbit Bridge from CHECK to KEYS, as both later codings read it: seven of its ten signers signed releases that no deposit backed, though the operator denied that keys were taken. 37 failures carry a borderline tag, mostly light clients whose verification is the receiving check (coded CHECK). For 5 of them a EVENT reading is also possible, and for 2 EVENT failures another reading; moving all of them gives the range of the verifier-side ceiling in Section 4. Minted value is valued at mint time at the backed asset's price, at the proceeds of the sale, or as reported, whichever the sources allow. Release times come from the chains wherever the releases could be traced; the dataset contains no addresses and no transaction sequences.

Robustness. Table 10 repeats the replay with the response times doubled or delayed, with only the failures whose first response was a pause, with each failure weighted equally, and without the largest failure. Its matched rows size one limit to Wormhole's combined traffic over 12 or 24 months and apply it to every failure, with and without a review of large transfers; the busiest four hours of the 24 months released $120 million against $30.9 million in the last 12, so the limit matched to 24 months lets out more and contains less.

Appendix C. Traffic and stake data

CCIP. We took the configuration of every token lane from the documentation repository of Chainlink at its commit of 14 October 2026, its history from the repository's commits and on-chain reads [6], and the transfers from the CCIP API [47]. The 18 lanes are the 15 that carried the most value from January to October 2026, with at most three lanes per token, plus the largest USDC lane, the largest LINK lane and the lane with the longest record of a binding limit. The USDC lane has no bucket, and one lane has less than the 120 days of history the schedule needs, which leaves 16 lanes in Table 6. Values are in US dollars at DefiLlama prices [48]. For transfers of values $v_1, \ldots, v_n$ at times $t_1 \le \cdots \le t_n$ and a refill r, the smallest capacity that delays no transfer is $B(r) = \max_{i \le j} [\sum_{k=i}^{j} v_k - r(t_j - t_i)]$, which one pass computes. B is convex, and we minimize $B(r) + rD$ by golden-section search. Replays queue transfers in order of arrival; the schedule refits every 30 days on the previous 90 days, chooses μ on the first half of each lane's segments, and starts each segment with a full bucket.

Wormhole. We built hourly series of the value leaving each of nine chains through Wormhole's token bridge from the Wormholescan API [49], with per-transfer records for the hours that contain transfers above the Governor's size threshold [8]. We removed transfers of Circle's CCTP, which the API lists under the token bridge but the Governor does not meter, and the transfers of a 2022 failure. The envelope $E(u)$ is the maximum over all start hours, so the replays round each response time up to whole hours, which can only overstate the loss.

Failed bridges. For each failed bridge we read every token transfer and value-bearing call out of its release contracts over the 150 days that end one second before the first unauthorized release. We kept those that match a release event of the contract in the same transaction, or all of them where the custody is an account that emits no events, dropped transfers to the bridge's own addresses, and priced each release at DefiLlama's price of its hour, or of its day where no hourly price exists. The three outsized releases before the Ronin and Harmony failures went to accounts that public address labels assign to an exchange, or that an exchange funded, not to the operators. Where custody is an account that emits no events, operator transfers to addresses we could not identify stay in the series.

Stake. We read stakes, and the balances Table 8 marks as on-chain, at Ethereum block 26,201,369 and Axelar height 36,297,293, both on 16 October 2026, and valued them at DefiLlama prices of the same hour. The other balances are the dollar totals the table's sources reported that day.